

文档密级：非密

课题名称：RISC-V 核心开发组件

RISC-V 核心开发组件

总体设计说明书

(版本：RuyiSDK 22.12)

编 制：陈小欧 陈小欧
审 核：邢明杰 邢明杰
批 准：邢明杰 邢明杰

完成日期：2022 年 11 月

中国科学院软件研究所

修订历史：

修订表					
版本号	修订描述	修订人	修订日期	审核人	审核日期
V1.0	新建文档	陈小欧	2022.11.10		
V1.1	添加V8 2022内容	邱吉	2022.11.18	邢明杰	2022.11.20

目 录

一、 文档概述.....	1
1.1 文档目的和范围	1
1.2 引用文件	2
1.3 名词术语解释	2
二、 设计概述.....	3
2.1 GCC 设计概述.....	3
2.2 LLVM 设计概述	9
2.3 V8 设计概述	18
2.4 OPENJDK 设计概述.....	21
2.5 QEMU 设计概述	25
2.6 OPENCV 设计概述.....	33
三、 总体设计.....	36
3.1 GCC 总体设计.....	36
3.2 LLVM 总体设计	41
3.3 V8 总体设计	44
3.4 OPENJDK 总体设计	51
3.5 QEMU 总体设计	57
3.6 OPENCV.....	67
四、 附录.....	74

一、文档概述

1.1 文档目的和范围

本文档旨在为 RISC-V 核心开发组件这一开发项目，提供一份全面、系统的总体设计说明。本文档的目的包括明确项目目标与边界，为所有项目干系人（开发团队、质量保证、管理层）提供统一的愿景和期待；阐述系统架构与设计决策，提供高层次的系统架构概览，描述各核心模块的职责、交互关系以及关键的技术选型与决策理由，为后续的详细设计、编码实现和维护奠定坚实的基础；指导开发与测试工作，为软件开发工程师、测试工程师提供实现和验证的依据，确保开发活动有序、高效地进行，并最终产出符合设计要求、高质量的代码；促进沟通与协作，作为团队内部与外部沟通的技术基准，确保信息传递的一致性和准确性，提升协作效率。

本文档涵盖对 RISC-V 核心开发组件如下四大任务的总体设计说明：

1. 面向 RISC-V 系统软件的编译工具支持和优化
2. 面向 RISC-V 应用软件的开发工具支持和优化
3. 面向 RISC-V 并行软件的开发工具支持和优化
4. 面向 RISC-V 软件开发的模拟验证平台

本文档所涉及的 RISC-V 扩展指令集包含如下几项，但不限于此：

- 1.B 扩展：提供循环移位、位域放置/提取、人口计数等高效位操作指令，用于提升加密、编码和底层软件的性能。

2. K 扩展：提供执行 AES、SHA、SM4 等标准密码学算法的专用指令，旨在为加密解密和哈希计算带来硬件级的加速。
3. P 扩展：提供一组用于 DSP 和嵌入式场景的 Packed-SIMD 指令。
4. Zce 扩展：通过提供表跳转、压栈/弹栈、16 位压缩指令等特性，旨在显著减少嵌入式设备的代码体积。
5. Zfinx 扩展：一种特殊的浮点编程模型，规定浮点指令操作于整型寄存器而非独立的浮点寄存器，简化了低成本嵌入式处理器的设计。

1.2 引用文件

1. 《先导科技项目任务书-课题 2-1》
2. 《RISC-V 核心开发组件需求规格说明书（22.12）》

1.3 名词术语解释

RISC-V 指令集架构：一种基于精简指令集计算（RISC）原则的开源指令集架构（ISA），主要特点是其开源性、可定制性和高度可扩展性；

RISC-V 指令集扩展：RISC-V 指令集架构由基础整数指令集和各种扩展指令集组成，扩展指令集分为标准扩展指令集和非标准扩展指令集；

RISC-V 核心开发组件：涵盖编译器、模拟器和运行时/虚拟机等；

编译器：主要负责将源代码编译、汇编和链接为目标指令集实现的二进制代码；

语言运行时/虚拟机：为解释型编程语言开发的软件提供开发环境，确保软件能够在 RISC-V 架构的设备上运行；

模拟器：为开发者提供仿真运行环境，使得开发者不受硬件设备的限制。

二、设计概述

2.1 GCC 设计概述

2.1.1 目标需求概述

(1) 核心目的：

开发完善一套支持 RISC-V 指令集架构的 GNU Toolchain

GNU Toolchain 是一套完整的程序开发工具集合，用于将高级语言源代码转换为目标平台的可执行程序或库，并支持调试、优化和系统级开发，其对传统架构如 X86-64, ARM 的支持已趋于完善，但对于 RISC-V 新兴架构的支持还有待补充。其主要组件包括：

GCC (GNU Compiler Collection)：多语言编译器（支持 C/C++/Fortran 等多种高级程序语言）。

Binutils：汇编器（as）、链接器（ld）、目标文件工具（objdump、readelf）。

GDB (GNU Debugger)：源代码级调试工具。

Glibc/Newlib：标准 C 库实现（分别用于 Linux 和嵌入式系统）。

(2) 核心目标：

在 GNU Toolchain (GCC + Binutils + GDB + Glibc/Newlib) 中实现对 RISC-V 指令集架构 (ISA) 的完整支持, 确保:

指令集兼容性: 正确生成 RV32I/RV64I 基础指令集及已批准的扩展指令集 (M/A/F/D/C/P/Zfinx/Zce/)。

优化代码生成: 利用 RISC-V 特性 (如压缩指令、SIMD) 提升性能或减少代码体积。

跨平台工具链: 支持主机 (x86-64/ARM) 到 RISC-V 目标的交叉编译与调试。

系统级开发支持: 适配裸机 (bare-metal)、Linux 内核及用户态上的应用开发。

2.1.2 运行环境

GCC 运行环境包括多个层次: 构建平台 (build)、主机平台 (host) 和目标平台 (target), 以及其运行所依赖的操作系统、工具和库, 下面从不同维度来介绍其运行环境和可能的系统局限性。

GCC 包括多个前端和后端组件, 关键的运行环境依赖包括: GCC 前端 (如 C/C++/Fortran) 处理如 bison, flex 工具, GCC 后端 (目标架构特定代码生成器, 如 RISC-V、x86) 处理工具 gawk, 汇编器 (gas), 链接器 (ld), 标准 C 库 (如 glibc 或 newlib), Binutils 二进制工具包, make、bash、perl、python 等工具 (用于构建), libstdc++: C++ 编译时所需的运行时库。一般在类 Unix 系统 (Linux, macOS) 上开发和使用, GCC 构建系统假设操作系统支持 POSIX 接口 (如 fork/exec、文

件系统、shell 命令等), 编译目标程序时, glibc 的版本需与运行环境兼容, 新版本的 GCC 编译的程序可能由于库函数处理不一致, 导致不能在旧版系统上正常运行。环境路径使用 /usr, /lib, /usr/include 等标准路径, 非标准系统可能需要配置 --prefix 和 --with-sysroot. GCC 和 Binutils 的版本需要匹配, 如果一方版本过旧, 在生成汇编时可能会出现兼容性问题(比如对特定 ISA 扩展的支持不一致, RISC-V 特性支持不一致等), 编译 GCC 本身资源消耗较高需要提供足够的资源(4GB+内存、数 GB 磁盘空间, 几十分钟至几小时构建时间)。

2.1.3 限制和约束

本软件/系统在开发与部署过程中, 受以下若干条件的限制与约束, 这些因素可能影响开发进度、功能实现范围、系统性能或最终交付形式。

(1) 技术条件

平台依赖性: 本系统需运行在指定操作系统(如 Linux x86_64), 因此所有代码必须兼容该平台的 API 与系统调用规范。

语言与框架限制: 项目使用 C/C++ 语言及 GNU 工具链(如 GCC、Make), 这限制了对其他现代开发语言(如 Rust、Go)的应用。

架构支持范围: 若涉及交叉编译(如 RISC-V、ARM), 需要依据目标平台架构特性调整优化策略, 部分功能可能无法跨平台完全复现。

第三方库/工具依赖: 如 glibc、Binutils、GDB 等版本需匹配, 否则会引发兼容性问题或功能不可用。

(2) 资源限制

人力资源：开发团队规模有限，人员技术覆盖面决定了项目中对复杂特性（如高级优化、自动化测试）的支持程度有限。

硬件资源：测试与部署依赖特定硬件（如嵌入式开发板、模拟器、FPGA 平台），数量有限可能导致并行开发受限。

构建资源：GCC 等大型项目编译时间长、资源占用大，对服务器或 CI 构建环境提出较高内存和存储要求（如至少 8GB RAM 和 10GB 硬盘空间）。

预算约束：可能无法采购高性能设备或商业编译器许可证，仅限使用开源或已有工具资源。

(3) 开发工具和开发平台、开发技术

开发平台：本项目主要在 Linux(如 Ubuntu 22.04)环境中开发，工具链依赖组件包括 GNU Make、Autotools、GCC、GDB 等。

版本控制系统：使用 Git 管理代码，但在某些合作单位/场所中受网络或访问策略限制，可能无法访问远程仓库。

CI/CD 工具限制：若使用 Jenkins、GitHub Actions 等自动化工具，需确保环境配置与安全策略符合目标平台部署要求。

编译工具链限制：特定目标平台的 GCC 或 LLVM 后端支持程度不同，部分实验性 ISA 扩展或优化策略（如 RISC-V Z*扩展）尚不稳定。

调试/测试工具约束：依赖 GDB 等工具进行调试和性能分析，但可能受限于目标系统环境（如裸机系统不支持传统调试方式）。

2.1.4 设计原则和设计要求

为确保 RISC-V GNU Toolchain 的可扩展性、稳定性与适配能力，本系统在设计过程中遵循以下关键设计原则：

(1) 模块独立性原则

功能分层、职责清晰：整个工具链按照功能进行模块化拆分，包括前端（GCC Frontend）、中间端（GIMPLE/RTL 优化器）、后端（目标指令生成器）、链接器（LD）、汇编器（AS）、调试器（GDB）等。

低耦合、高内聚：模块之间通过明确接口进行交互，避免模块间直接访问内部数据结构，提高系统稳定性与可替换性。

独立构建：每个组件（如 GCC、Binutils、Newlib）可以独立编译测试，符合多版本适配需求。

(2) 边界设计原则

架构边界清晰：目标平台与主机平台解耦，支持交叉编译；工具链可同时支持 multiple target triples(如 riscv64-unknown-elf 与 riscv64-linux-gnu)。

接口封装明确：如 libgcc、libstdc++ 提供标准接口以供上层调用，不暴露底层实现细节。

ISA 扩展分层封装：针对不同的 RISC-V 子集（如 RV32/64i,Zb*/Zc*/Z*inx），通过后端 target hook 控制支持范围和优化策略，尽量避免耦合在核心逻辑中。

(3) 数据库设计原则

本系统不直接使用数据库，但

内部信息管理：使用结构化中间表示（IR）如 GIMPLE、RTL 等作为“数据层”，统一维护符号表、依赖关系与优化信息。

编译信息持久化：如.o 文件、.a 库、.debug 段存储编译中间结果，逻辑上可视为文件型数据库，需保持结构一致、可读性强。

(4) 灵活性要求

多目标支持：设计必须支持灵活切换 ABI 依赖和 ISA 选项（如-march,-mabi），同时支持 soft-float、hard-float、SIMD、bitmanip 等变种。

可扩展性强：允许用户通过添加新的 target backend（如新的 RISC-V 扩展）或 frontend（如 Rust frontend 通过 LLVM 插件）进行扩展。

配置驱动：多数功能通过 configure 脚本和--enable-*/--with-*参数控制，避免硬编码。

(5) 易操作性要求

一致的命令行接口：工具链的各子模块（gcc、ld、as、objdump、gdb）遵循 GNU 工具链的标准语法，便于开发者能够快速上手。

文档友好：提供完整的 man pages、info 文档和 online docs，便于查询参数和行为。

标准化输出：符合 ELF、DWARF、GAS 语法等工业标准，确保用户编写的程序可被广泛工具链识别和调试。

(6) 可维护性要求

社区协作友好：工具链的开发遵循 GNU 项目代码风格和提交流程，易于维护者和外部贡献者共同参与开发。

注释清晰、代码分层：源代码中提供详细注释，复杂逻辑分层抽象，便于维护者定位问题。

版本化与兼容性管理：通过发行版本标识（如 GCC 15.1、Binutils 2.44），并保持向后兼容的选项和行为，便于长期维护与用户升级。

测试覆盖广泛：工具链包含 DejaGNU、GCC testsuites 等大量自动化测试，提升代码变更的稳定性保障。

2.2 LLVM 设计概述

2.2.1 目标需求概述

LLVM（Low Level Virtual Machine）是一个开源的编译器基础设施项目，旨在提供模块化、可重用的编译器与工具链技术。它采用现代化的设计理念，将前端（如 Clang）、优化器和后端分离，支持多种编程语言（如 C/C++、Rust、Swift）和硬件架构（如 x86、ARM、GPU）。LLVM 的核心是其高效的中间表示（IR），允许开发者进行跨平台的代码优化与生成。凭借其灵活性、高性能和活跃的社区，LLVM 已成为现代编译器开发（如 Clang、Rustc）和 JIT 编译（如 Julia）的核心引擎，广泛应用于操作系统、嵌入式系统和高性能计算领域。

LLVM 添加对 RISC-V 扩展指令集的支持主要涉及后端模块的扩展，这一过程需要在编译流程和汇编处理两个关键环节进行协同优化。在编译层面，LLVM 需要能够识别并处理 RISC-V 的各种扩展指令集，包括正确解析指令语义、优化寄存器分配策略，以及根据目标平台特性进行指令调度和代码优化。在汇编层面，系统需要支持扩展指令的

汇编与反汇编功能，确保能够正确生成和解析包含扩展指令的目标代码，同时保持与标准工具链的兼容性。本部分主要涉及汇编层面的工作，主要包括：在汇编模块中添加对 RISC-V B 扩展的汇编支持，在汇编模块中添加对 RISC-V K 扩展的汇编支持，在汇编模块中添加对 RISC-V P 扩展的汇编支持，在汇编模块中添加对 RISC-V Zce 扩展的汇编支持和在汇编模块中添加对 RISC-V Zfinx 扩展的汇编支持。

2.2.2 运行环境

LLVM 的运行环境支持多种操作系统，包括 Linux、macOS、Windows 以及嵌入式系统。它对硬件架构的覆盖也非常广泛，涵盖 x86、ARM、RISC-V 等通用架构，以及 NVIDIA 和 AMD 的 GPU。依赖工具链包括 CMake（构建系统）、支持 C++17 的编译器（如 GCC 或 Clang）、Python（用于测试工具）以及可选的 Ninja（加速构建）。这些依赖确保了 LLVM 可以在不同平台上顺利编译和运行。

安装 LLVM 可以通过预编译二进制包或源码编译完成。预编译版本适合快速部署，而源码编译允许自定义组件和优化选项。配置环境变量（如将 LLVM 工具链添加到 PATH）是使用 LLVM 的关键步骤，确保命令行工具（如 clang、lldb）可以直接调用。源码编译时，通常需要指定目标组件（如 Clang、LLD）和安装路径。

LLVM 提供丰富的工具链和实用工具，包括编译器驱动（clang、clang++）、调试器（LLDB）、IR 优化工具（opt）、目标代码生成器（llc）以及二进制分析工具（llvm-objdump）。这些工具覆盖了从代码编译到

调试、优化的全流程。此外，Clang-Format 和 Clang-Tidy 等工具支持代码格式化和静态分析，帮助开发者提高代码质量。

LLVM 的跨平台支持非常强大，支持交叉编译（通过 `-target` 参数指定目标平台）和多种运行时库（如 `Compiler-RT`、`libc++`）。开发者可以轻松地为不同硬件生成代码，无论是嵌入式设备还是高性能服务器。LLVM 的运行时库提供了底层支持，确保生成代码的可靠性和性能。

LLVM 支持动态（JIT）和静态（AOT）编译。JIT 编译通过 LLVM ORC 或 MCJIT 实现，适用于解释型语言或运行时优化；AOT 编译生成静态可执行文件，适合传统应用。这种灵活性使 LLVM 能够适应多种场景，从即时执行的脚本语言到高性能计算。

LLVM 可以容器化部署，官方提供 Docker 镜像（如 `llvm/clang`），方便在云环境或 CI/CD 中使用。云 IDE（如 VS Code Remote、GitHub Codespaces）也支持 LLVM 开发环境的快速配置，适合团队协作和远程开发。

2.2.3 限制和约束

本软件/系统在开发与部署过程中，受以下若干条件的限制与约束，这些因素可能影响开发进度、功能实现范围、系统性能或最终交付形式。

(1) 技术条件

LLVM 在硬件架构支持方面虽然广泛覆盖 x86、ARM、RISC-V

等主流架构，但对新兴或专用架构（如某些 DSP 或 AI 加速器）的支持往往滞后，且不同后端的代码生成质量存在差异，例如 RISC-V 的优化可能不如 x86 成熟，而 GPU 代码生成需要依赖厂商特定的工具链（如 NVIDIA 的 NVPTX），这限制了其在异构计算场景的即插即用能力。

在编译器工具链依赖方面，LLVM 自身作为用 C++17 编写的项目，对构建环境有严格要求，需要较新版本的 GCC、Clang 或 MSVC，且完整构建需要消耗大量计算资源（通常需要 16GB 以上内存和数十 GB 磁盘空间），这使得在资源受限的嵌入式开发机上编译 LLVM 工具链变得极具挑战性，同时也阻碍了其在老旧系统上的部署。

操作系统兼容性方面，虽然 LLVM 官方支持 Linux、macOS 和 Windows，但在不同平台上的功能完备性并不一致，例如 Windows 平台上的调试符号处理能力和对 MSVC 的兼容性长期落后于类 Unix 系统，而嵌入式操作系统（如 FreeRTOS）通常需要额外的移植工作来支持 LLVM 运行时库，这些差异增加了跨平台开发的复杂度。

性能优化方面，LLVM 虽然提供从 -O0 到 -O3 的多级优化，但高级优化往往以显著增加的编译时间为代价，在处理大型项目时可能使增量构建变得不切实际，同时其内存占用随着优化级别提升而急剧增长，这对持续集成环境和开发者本地机器都提出了较高的硬件要求，需要在优化收益和开发效率间谨慎权衡。

标准库和运行时依赖方面，LLVM 生态存在碎片化问题，其自研的 libc++ 与 GNU 的 libstdc++ 不完全兼容，可能导致 C++ 程序的 ABI

问题，而 `compiler-rt` 等运行时库的部署也需要特别注意，在交叉编译时容易因链接错误导致生成的可执行文件无法在目标平台正常运行，这些问题在嵌入式开发中尤为突出。

跨平台编译的复杂性体现在需要精确配置目标三元组和系统库依赖，开发者不仅要处理不同平台的调用约定差异，还要解决系统头文件和库的版本兼容性问题，例如在 `macOS` 上为 `Linux` 目标编译时需要避免链接到 `Darwin` 专属库，而不同 `Linux` 发行版之间的 `glibc` 版本差异也会导致二进制兼容性问题，这些细微差别大大增加了交叉编译的调试难度。

调试和工具链整合方面，`LLVM` 虽然提供 `LLDB` 等现代化工具，但与传统调试器（如 `GDB`）的兼容性仍有不足，`DWARF` 调试信息格式的支持程度在不同工具间存在差异，同时将 `LLVM` 工具链集成到非 `CMake` 构建系统（如 `Bazel` 或 `Makefile`）需要大量的适配工作，这些因素都提高了项目迁移到 `LLVM` 生态的技术门槛。

在许可和法律约束方面，虽然 `LLVM` 采用相对宽松的 `Apache 2.0` 许可证，但其专利条款与 `GPL` 许可证的兼容性存在争议，可能影响某些开源项目的采用决策，同时部分功能（如 `Windows COFF` 格式的完整支持）受限于第三方专利，导致相关实现（如 `LLD` 链接器）在某些场景下可能无法完全替代专有工具链，这些法律因素在实际部署时需要仔细评估。

(2) 资源限制

`LLVM` 在开发和部署过程中面临的资源限制主要体现在计算资

源消耗、存储需求、内存占用、带宽依赖和人力资源投入等方面，这些限制直接影响开发效率和部署成本。

计算资源消耗: LLVM 的编译和优化过程对 CPU 性能要求较高，尤其是启用高级优化（如 -O3 或 LTO）时，编译时间可能大幅增加。大型项目（如 Chromium 或 Linux 内核）的完整构建可能需要数小时，对 CI/CD 流水线和开发者的本地开发环境造成压力。此外，JIT 编译（如使用 ORC 或 MCJIT）在运行时也会引入显著的 CPU 开销，影响程序启动速度。

内存占用: LLVM 的优化器和代码生成器（如 `opt` 和 `llc`）在处理复杂代码时可能占用大量内存，尤其是在进行过程间优化（IPO）或链接时优化（LTO）时，内存消耗可能达到数十 GB。例如，使用 ThinLTO 编译大型 C++ 项目时，可能需要 64GB 甚至更高容量的 RAM，否则可能导致编译失败或性能下降。

存储需求: LLVM 工具链本身占用较大磁盘空间，完整构建（包括 Clang、LLDB、编译器运行时库等）可能需要 50GB 以上的存储空间。此外，调试信息（如 DWARF 格式）和中间文件（如 LLVM IR 或 Bitcode）会进一步增加存储负担，尤其是在多配置（Debug/Release）或多目标（交叉编译）开发场景下。

网络带宽依赖: 从源码构建 LLVM 需要下载较大的代码仓库（LLVM 项目整体超过 1GB），且依赖第三方库（如 `zlib`、`libxml2` 等）。在 CI/CD 环境中，频繁的干净构建可能导致大量带宽消耗，特别是在云服务器或远程开发机上部署时。

人力资源投入：LLVM 的学习曲线较陡，开发者需要熟悉其模块化架构、IR 优化机制和目标后端开发方式。定制化开发（如编写自定义 Pass 或支持新硬件架构）需要深入的编译器知识，增加了团队的技术培训成本。此外，调试 LLVM 生成的代码（尤其是优化后的 IR 或机器码）比调试普通高级语言更复杂，需要额外的时间和经验。

嵌入式环境限制：在资源受限的嵌入式设备上部署 LLVM 工具链时，可能面临内存不足、存储空间有限、缺少动态链接支持等问题。例如，某些微控制器（如 Cortex-M 系列）的 Flash 和 RAM 容量较小，难以运行完整的 LLVM 优化流程，通常需要裁剪编译器功能或使用预编译的运行时库。

云环境与容器化限制：在容器化部署（如 Docker）或云服务器上运行 LLVM 时，可能受限于 CPU 配额、内存限制和临时存储空间。例如，某些 CI/CD 平台（如 GitHub Actions）对单次任务的内存和运行时间有限制，可能导致大型项目的 LLVM 编译失败。

(3) 开发工具和开发平台、开发技术

LLVM 在开发和部署过程中涉及多种开发工具、平台和技术，涵盖编译器构建、代码优化、调试、测试和跨平台支持等方面。

LLVM 的核心开发工具主要包括 Clang（C/C++ 前端编译器）、Flang（Fortran 前端）以及一系列 LLVM IR 处理工具（如 opt 优化器、llc 代码生成器、lli 解释器）。链接环节使用 LLD 高性能链接器或 wasm-ld（WebAssembly 专用），这些工具共同构成了完整的编译工具链。调试分析方面，LLDB 提供类似 GDB 的调试功能，配合 AddressSanitizer

等检测工具可进行内存错误分析，而 Clang-Format 和 Clang-Tidy 则负责代码风格检查和静态分析。

开发平台支持非常广泛，原生支持 Linux、macOS 和 Windows 三大操作系统，并能适配各类 BSD 系统和嵌入式 RTOS 环境。硬件架构支持涵盖主流 CPU(x86/ARM/RISC-V)和 GPU(NVIDIA/AMD)，通过不同的后端实现跨平台编译。云环境部署时，官方 Docker 镜像和 GitHub Codespaces 提供了便捷的容器化开发方案，Kubernetes 则可管理分布式编译任务。

在开发技术层面，CMake 和 Ninja 构成了主要构建系统，支持高效的跨平台编译。优化技术包括 LTO 链接时优化、PGO 基于性能分析的优化以及 BOLT 优化。跨平台编译通过目标三元组 (Triple) 机制实现，配合 Sysroot 配置可精确控制目标环境。运行时环境由 libc++ 标准库、compiler-rt 内置函数支持和 libunwind 异常处理共同构建，而 ORC JIT 和 MLIR 则提供了动态编译和领域特定优化的能力。

测试验证环节采用 Lit 测试框架和 FileCheck 验证工具，结合 LLVM Test Suite 进行全面的正确性和性能测试。典型开发流程从代码编辑开始，经过构建、分析、调试、优化等阶段，最终通过交叉编译或容器化完成部署。整个技术栈的设计既保证了 LLVM 的强大功能，也带来了较高的学习和使用门槛，开发者需要根据具体场景选择合适的工具链和优化策略。

2.2.4 设计原则和设计要求

LLVM 的设计遵循一系列核心原则，使其成为高效、灵活且可扩展的编译器框架：

(1) 模块化与分层设计

LLVM 采用模块化架构，将编译器分为前端（源码转 IR）、优化器（IR 优化）和后端（IR 转机器码），各阶段可独立替换或扩展。这种分层设计使得支持新语言或新硬件只需实现对应模块，无需重写整个编译器。

(2) 统一的中间表示

LLVM IR 是核心抽象，兼具高级语言的可读性和低级语言的精确性，便于跨语言优化。IR 采用静态单赋值（SSA）形式，简化数据分析，同时支持文本和二进制格式（Bitcode），便于存储和传输。

(3) 优化驱动

LLVM 强调编译时优化，提供丰富的优化 Pass（如内联、循环展开、死代码消除），支持基于 Profile 的优化（PGO）和链接时优化（LTO），确保生成代码的高效性。

(4) 目标无关与目标相关分离

大部分优化在 IR 层完成（目标无关），后端仅处理机器相关优化（如指令选择、寄存器分配）。这种分离简化了新架构的支持，例如 RISC-V 后端只需实现少量机器相关逻辑即可复用现有优化器。

(5) 工具链整合

LLVM 不仅是一个编译器，还提供完整工具链（Clang、LLD、

LLDB)，强调工具间的协同性。例如，调试信息（DWARF）在 IR 阶段生成，可被优化器保留并在后端映射到机器码。

(6) 运行时灵活性与可扩展性

支持静态编译(AOT)、即时编译(JIT)和动态加载(如 ORC JIT)，适应不同场景。MLIR 的引入进一步扩展了 LLVM 的领域，支持自定义中间表示（如 TensorFlow 的算子优化）。

LLVM 的设计要求聚焦于构建一个高性能、可扩展且跨平台的编译器基础设施，其核心在于通过模块化架构实现前端、优化器和后端的解耦，确保各组件可独立开发和替换；采用统一的 LLVM IR 作为中间表示，兼具人类可读性和机器可优化性，支持静态单赋值(SSA)形式以简化分析；强调优化驱动的设计哲学，提供丰富的优化 Pass 并支持基于性能分析的 PGO 和链接时优化 LTO；必须保持目标无关与目标相关逻辑的严格分离，使新硬件支持只需实现后端而复用通用优化器；要求完整的工具链集成（如 Clang、LLD、LLDB）和多场景适应性（AOT/JIT/动态加载）；同时需满足高性能编译（低内存/时间开销）、跨平台兼容性（支持多 OS/架构/ABI）、强可测试性（Lit 框架）和开发者友好性（文档/API 稳定性），最终形成既能支撑学术创新又能服务工业级应用的编译器生态系统。

2.3 V8 设计概述

2.3.1 目标需求概述

为了在 V8 引擎上支持 RISC-V32 和 64 位的基础指令集，需在 V8

与 Chromium 构建系统中添加 RISC-V 本地及交叉编译支持，实现 RISC-V 指令集架构下 V8 后端的代码生成、指令选择等核心模块开发，完成 RISC-V 64 位 IMFDA 指令集支持并通过模块测试、随机测试及回归测试后提交上游社区，基于此修改实现 RISC-V 32 位 IMFDA 指令集支持并完成对应测试与代码提交，同时实现 C 扩展指令集支持，基于 V8 内置模拟器建立 RISC-V 架构的 daily CI 以持续跟踪修复 bug。

2.3.2 运行环境

(1) 操作系统:

- Linux 操作系统，如 Debian/Ubuntu
- 依赖软件：V8 通过 depot_tools 来管理和维护项目的代码仓库、依赖关系以及相关的开发工作流程。

2.3.3 限制和约束

(1) 技术条件

RISC-V 的模块化设计（如 IMFD/A/B/V 等扩展）要求开发团队深入理解各指令集规范（如 RV64IMFDA 与 RV32IMFDA 的位宽差异、寻址模式），若缺乏 RISC-V 架构经验，易导致指令选择错误或性能损耗。

Chromium 和 Node.js 的第三方库（如 OpenSSL）对 RISC-V 的原生支持不足，需手动修改构建脚本或移植补丁，可能面临库版本兼容

问题（如 OpenSSL 的汇编优化代码需重写以适配 RISC-V 指令集）。

(2) 资源限制

上游社区（如 V8）的代码评审流程严格，需专人对接沟通，若团队缺乏开源协作经验，可能导致代码合并周期延长。代码合入后，需要专人长期维护，才能保证 RISC-V 的 V8 支持与 Tier-1 平台保持一致。

(3) 开发工具和开发平台、开发技术等

RISC-V 硬件平台较为稀缺，成本较高，依赖模拟器（如 Qemu 和 V8 内置模拟器）进行性能测试，则可能受限于模拟环境与真实环境的差别导致无法准确验证正确性和获取性能数据。模拟器性能不足（如 V8 内置模拟器对复杂指令的模拟速度慢），还可能导致测试覆盖率不足或耗时过长。

2.3.4 设计原则和设计要求

(1) 不同后端的最大复用原则

划分 V8 后端的指令选择、寄存器分配、代码生成模块的“体系结构通用”和“体系结构专用”代码和 API 边界，避免重复工作和代码冗余。

(2) 严格遵循 RISC-V 的应用二进制接口（ABI）规范（如 RV64 的 lp64d、RV32 的 ilp32d），确保与外部代码的互操作性。

(3) 扩展指令集的灵活配置：提供通用的扩展指令检测和注册机制，支持模块化的定制和集成。

2.4 OpenJDK 设计概述

2.4.1 目标需求概述

OpenJDK (Open Java Development Kit) 是 Java 平台标准版 (Java SE) 的官方开源实现, 由 Sun Microsystems 于 2006 年启动, 现由 Oracle 主导, 并依托全球开发者社区共同维护。它是 Java 生态系统的核心, 为 Java 应用程序提供运行环境 (JVM) 和开发工具 (如编译器、调试器等)。同时, OpenJDK 是 Java 生态的基石, 提供高性能 JVM、丰富的 API 和工具链。其开源模式促进了广泛采用, 各大厂商提供优化版本, 适用于从嵌入式到云计算的各类场景。

OpenJDK 要面向 RISC-V 指令集, 最终将落实在 RV64G 和 RV32G 上。通过对 RV64G 和 RV32G 的支持, 就可以支持绝大多数 RISC-V 指令集架构设备。OpenJDK 对于 RV64G 的支持, 已经在社区进行了展开, 但是对于 RV32G 的支持, 还未有实现。

本项目聚焦于 OpenJDK 对于 RISC-V 的支持, 具体的需求分为 RV64 和 RV32 两部分。RV64 部分, 要实现 OpenJDK RV64G 支持现状调研和 RV64G 开发板支持。OpenJDK RV64G 支持现状调研, 主要对于 OpenJDK 目前的开发版本, 特别是主干代码中, 对于 RV64G 指令集的支持情况, 做详细的调研。RV64G 开发板支持, 则是调研市场上目前已经量产的可以支持 RV64G 的开发板, 然后使用 OpenJDK 测试其支持和适配。RV32 部分, 则包含: OpenJDK 32 位版本调研、OpenJDK 32 位版本基线选择、OpenJDK RV32G 移植内容调研和

OpenJDK RV32G Zero 解释器移植。OpenJDK 32 位版本调研，是要调研目前市场上使用比较多的 OpenJDK 32 位版本，为后续选择移植基线做好准备。OpenJDK 32 位版本基线选择，需要比较目前的 OpenJDK 32 位版本，从中选择一个版本作为移植的基线版本，为后续移植工作做好准备。OpenJDK RV32G 移植内容调研，是要调研 OpenJDK RV32G 移植所需要修改的内容范围，包含新增的文件以及原有代码的修改，确定整个移植所涉及到的范围。OpenJDK RV32G Zero 解释器移植，针对 RISC-V RV32G，对 Zero 解释器进行代码调整和测试。

2.4.2 运行环境

本项目开发环境的操作系统为 Ubuntu 18.04 LTS，具体信息如下：

No LSB modules are available.

Distributor ID: Ubuntu

Description: Ubuntu 18.04.5 LTS

Release: 18.04

Codename: bionic

除了操作系统之外，开发环境还需要依赖一批相关的工具和软件，主要有：autoconf automake autotools-dev curl libmpc-dev libmpfr-dev libgmp-dev gawk build-essential bison flex texinfo gperf libtool patchutils bc zlib1g-dev libexpat-dev git libglib2.0-dev libfdt-dev libpixman-1-dev libncurses5-dev libncursesw5-dev ninja-build python3 autopoint pkg-config zip unzip screen make libxext-dev libxrender-dev libxtst-dev libxt-

dev libcups2-dev libfreetype6-dev mercurial libasound2-dev cmake
libfontconfig1-dev python3-pip gettext。

同时，还需要编译交叉工具链 `riscv-gnu-toolchain`。`riscv-gnu-toolchain` 可以通过两种方式获得，一种是通过源码编译，另外一种是直接获得已经编译好的可执行开发包。

此外，还需要 QEMU 来模拟 RV64G 和 RV32G 的硬件环境，通过该硬件环境来调试和运行 OpenJDK for RISC-V。也可以准备比较成熟的开发板，用来运行和验证 OpenJDK RV64G 和 OpenJDK RV32G。

2.4.3 限制和约束

本项目在开发过程中，需要满足诸多的条件和限制，主要有：

(1) 技术条件

代码规范：开发过程中应该遵守 Java 语言规范 (JLS) 和 JVM 规范，保证新增代码符合代码规范，和原有代码保持风格一致。

开发工具链依赖：选用 `riscv-gnu-toolchain` 作为交叉编译的工具链，从 X86 架构交叉编译为 RISC-V 架构。

构建系统：使用 Autoconf 和 Make 进行构建，部分模块（如有自定义的构建脚本，则使用自定义脚本。

平台支持：项目在满足 RISC-V 目标架构要求的同时，不应该破坏其他指令集架构的相关代码。

(2) 资源限制

资源限制：本项目在 RISC-V 开发板上的运行和调试，需要成熟

的有操作系统支持并且分别支持 RV32G 和 RV64 指令集的开发板。

(3) 开发平台

开发平台：本项目在 X86 架构的 Ubuntu 18.04 LTS 上进行开发和调试。

2.4.4 设计原则和设计要求

本项目为 OpenJDK 的移植项目，遵循 OpenJDK 的设计原则和设计要求，主要有：

(1) 模块独立性原则

OpenJDK 采用了平台模块化和代码分层设计，对 JVM 实现、核心库和工具链等部分内容，进行了分离，使得模块之间界限清晰，减少了运行时的依赖，并且方便开发者进行二次开发。

(2) 边界设计原则

OpenJDK 的设计不仅关注内部架构的合理性，还严格定义了系统与外部环境（如操作系统、硬件、第三方库、其他语言等）的交互边界。这些边界设计原则确保 OpenJDK 在保持灵活性和可扩展性的同时，避免过度耦合或不可控的外部依赖。主要包括了清晰的系统分层、平台抽象层、第三方依赖最小化、语言互操作边界、安全边界、网络与 I/O 边界等，这些边界设计原则可以大大提升 OpenJDK 的可移植性、安全性和可维护性。

(3) 灵活性要求

OpenJDK 的设计不仅需要保证稳定性、性能和安全性，还必须具

备足够的灵活性，以适应多样化的应用场景、技术演进和用户需求。本项目遵循 OpenJDK 的灵活性要求，主要包括：可配置性与可扩展性、模块化与动态加载和多语言与多范式支持等。

可配置性欲可扩展性，通过运行时的参数调优和 SPI 机制，采用丰富的 JVM 参数，使得用户根据自己的需求可以调整内存、GC、JIT 等一系列行为。模块化与动态加载，则支持按需加载模块，减小启动开销；并可以自定义类加载器，支持热部署和隔离。多语言与多范式支持，可以实现多语言互操作，实现函数式与响应式编程。

(4) 易操作性要求

OpenJDK 的设计不仅关注开发者的使用体验，还需要确保运维人员、系统管理员 等角色能够方便地监控、调试和优化 Java 应用程序。本项目遵循 OpenJDK 的易操作性要求，主要有：命令行工具友好性、日志与诊断信息可读性、监控与可视化支持、配置管理的便捷性、容器化与云原生适配、文档与社区支持、安装与升级的简易性等。

(5) 可维护性要求

本项目的可维护性要求，主要包括：代码结构与组织规范、构建规范的测试体系、做好代码变更管理、完善文档和知识库等。

2.5 QEMU 设计概述

2.5.1 目标需求概述

QEMU (Quick Emulator) 是一款开源的硬件虚拟化工具，支持全系统模拟 (Full System Emulation) 和用户模式模拟 (User-mode

Emulation)。它通过动态二进制翻译 (Dynamic Binary Translation, DBT) 技术, 能够跨平台模拟多种 CPU 架构 (如 x86、ARM、RISC-V、MIPS 等), 允许用户在一个硬件架构上运行另一种架构的操作系统和程序。

QEMU 的全系统模拟 (Full System Emulation) 功能支持广泛的 CPU 架构, 涵盖主流处理器、嵌入式芯片及新兴指令集。例如 x86/x86_64、ARM、PowerPC、RISC-V、MIPS、SPARC 等。全系统模拟的典型应用场景有跨平台开发, 例如在 x86 主机上运行 ARM/RISC-V 的 Ubuntu 进行测试; 固件调试, 例如模拟 MIPS 路由器环境分析漏洞; 历史系统保留, 例如运行 PowerPC 版的 Mac OS 9; 教育研究, 例如通过 RISC-V 模拟器学习计算机体系结构等。

QEMU 的全系统模拟功能中 RISC-V 架构的支持仍处于发展阶段, 部分扩展指令集的支持仍不完善或无支持, 本课题拟对 B、K、P、Zce、Zfinx 扩展指令集进行支持, 实现 QEMU 在全系统模拟中对 RISC-V 架构的 B、K、P、Zce、Zfinx 扩展指令集的支持。

本年度目标需求是实现 QEMU 在全系统模拟和用户模式模拟中对 RISC-V B 扩展指令集的支持。

2.5.2 运行环境

(1) 操作系统

QEMU 可以运行在多种操作系统上, 包括 Linux、Windows、MacOS 和其他类 Unix 系统, 例如 BSD。

(2) 依赖环境

编译工具链: GCC/Clang 用于编译 QEMU 源码, GNU Make/Ninja 用于构建, Autotools 用于生成构建配置, Python 3.x 用于代码生成、测试等。

库依赖: Glib 2.0 是 QEMU 的核心依赖, 提供数据结构、事件循环等基础功能。Libpixman 用于像素操作库, 用于图形渲染, zilib 用于数据压缩支持, SDL/GTK 用于图形界面支持, libvirt 用于虚拟化管理工具集成。

(3) RISC-V 相关工具

RISC-V 工具链 (RISC-V GCC, RISC-V Binutils) 用于交叉编译 RISC-V 目标程序。

RISC-V 仿真器 (Spike、QEMU 自身): 用于验证扩展指令的正确性。

2.5.3 限制和约束

(1) 技术限制

- QEMU 框架的约束

新增指令的实现必须基于 QEMU 的 TCG (Tiny Code Generator) 中间代码框架, 不能绕过其翻译和执行机制。

指令解码需遵循 QEMU 现有的 decodetree 规则 (insn32.decode/insn16.decode 文件格式)。

不支持直接修改 QEMU 的底层硬件模拟逻辑 (如内存模型、中断控制器), 必须通过 RISC-V 标准 CSR (控制状态寄存器) 接口扩

展功能。

- RISC-V 标准兼容性

新增指令必须符合 RISC-V 官方指令集手册（ISA Manual）的规范（如特权架构 v1.12、向量扩展 v1.0 等）。

若为自定义扩展（非标准指令），需明确其与标准指令的冲突规避方案（例如保留操作码空间）。

- 性能限制

指令模拟的性能不得显著低于 QEMU 现有 RISC-V 基础指令的实现（例如单条扩展指令的 TCG 翻译开销需控制在合理范围内）。

(2) 开发环境约束

- 平台依赖性

代码必须能在 Linux/gcc 和 Windows/MSVC 环境下编译通过，确保跨平台兼容性。

仅支持 GLib 2.0+ 和 Python 3.x 作为核心依赖，避免引入额外第三方库。

- 工具链依赖

需依赖 RISC-V 工具链（riscv-gcc/llvm）生成测试用例，但不得强制要求用户安装特定版本工具链（需提供预编译的测试二进制文件）。

(3) 资源与时间约束

- 人力资源

开发团队需具备 QEMU 内部机制和 RISC-V 指令集架构的深入

知识，初期学习成本较高。

- 测试资源

缺乏完整的 RISC-V 硬件验证环境，需依赖 Spike 模拟器或 FPGA 实现 进行交叉验证。

测试用例需覆盖指令的 边界条件（如异常输入、特权级切换），但受限于 QEMU 的调试工具支持。

(4) 兼容性约束

- 向后兼容性

新增指令不得破坏 QEMU 现有 RISC-V 模拟功能（如 RV64GC 基础指令集、特权模式、设备树支持）。

需确保与主流 RISC-V 操作系统（如 Linux、FreeRTOS）的兼容性，避免因指令实现差异导致内核崩溃。

- 向前扩展性

代码需预留接口，支持未来可能的指令集扩展（例如通过宏或条件编译隔离实验性功能）。

(5) 法律与许可约束

- 开源协议

代码必须遵循 QEMU 的 GPLv2 许可证，任何新增文件或修改需明确标注版权和协议声明。

禁止引入闭源或协议冲突的第三方代码（如某些商业指令集扩展的专利实现）。

- 专利规避

若扩展指令涉及专利技术（如某些向量指令优化方案），需确保其已开放授权或属于 RISC-V 基金会免专利范围。

(6) 文档与维护约束

● 文档要求

必须提供完整的开发者文档，包括指令解码逻辑、TCG 操作映射、CSR 变更说明。

在 QEMU 官方 Wiki 或 docs/devel/ 目录下提交文档更新，确保与主分支同步。

● 代码维护

代码风格需严格遵循 QEMU 的编码规范（如缩进、命名规则、提交日志格式）。

需通过 QEMU 社区的代码审查流程，合并前需通过 `make check-tcg` 和 `make check-acceptance` 测试集。

2.5.4 设计原则和设计要求

(1) 模块独立性原则

● 高内聚低耦合

RISC-V 扩展指令的实现需封装在独立的模块中（如 `target/riscv/` 下的 `insn_translate.c`、`cpu_helper.c`），避免与 QEMU 其他架构代码（如 x86、ARM）直接耦合。

指令解码、执行、异常处理逻辑应分离，通过清晰接口交互（如 `decode_opcode() → execute_insn()`）。

- 动态扩展支持

通过宏或条件编译（如`#ifdef TARGET_RISCV_VEXT`）支持未来指令集的灵活启用/禁用。

(2) 边界设计原则

- 硬件与模拟器边界

严格遵循 RISC-V ISA 手册定义的指令行为，不模拟非标准硬件特性（如特定厂商的微架构优化）。

虚拟设备（如 CLINT、PLIC）的交互需通过 QEMU 的设备树（DTB）和内存映射 I/O 机制实现。

- 特权级隔离

用户模式（U-Mode）与监督模式（S-Mode）的指令行为需严格区分，避免权限泄露。

(3) 状态管理原则

- CPU 状态一致性

扩展指令对 CPU 状态（如寄存器、CSR）的修改需通过 `cpu_restore_state()` 等接口同步，确保快照(snapshot)和迁移(migration)功能正常。

- 原子操作支持

若扩展指令包含原子操作（如 AMO），需依赖 QEMU 的 `atomic.h` 机制保证多线程安全。

(4) 灵活性要求

- 配置化支持

通过 QEMU 命令行参数（如 `-cpu rv64,vext=true`）动态启用/禁用扩展指令集。

- 多平台适配

代码需兼容 QEMU 的系统模式（`full-system`）和用户模式（`linux-user`）仿真。

(5) 易操作性要求

- 调试支持

提供 `-d exec,in_asm` 日志输出，实时跟踪指令解码与执行流程。
支持 GDB 远程调试（通过 `-s -S` 参数暴露调试接口）。

- 文档化接口：

在 `docs/devel/riscv-extensions.md` 中明确扩展指令的使用方法和示例。

(6) 可维护性要求

- 代码规范

遵循 QEMU 的编码风格（如 4 空格缩进、函数命名 `riscv_insn_*`）。
提交代码前需通过 `scripts/checkpatch.pl` 静态检查。

- 测试覆盖

单元测试需覆盖指令的正常路径和异常路径（如非法操作码、权限不足）。

集成测试需通过 QEMU 的 `make check-tcg` 测试套件。

2.6 OpenCV 设计概述

2.6.1 目标需求概述

本项目旨在重构 OpenCV 的通用向量计算框架（Universal Intrinsic），使其原生支持 RISC-V RVV 可变长向量架构，彻底消除传统固定长度 SIMD 的数据填充与转换瓶颈。核心需求的技术转化包括：

- (1) **动态硬件适配**：通过运行时读取硬件寄存器，实时获取向量长度和元素位宽，动态调整数据加载策略，实现循环展开与尾端截断的自动化管理。
- (2) **多版本兼容**：设计指令集隔离层，支持最新版本的 RVV v1.0 版本指令集，抽象 GCC/Clang 工具链的语法差异，确保跨编译器兼容性。
- (3) **性能优化**：基于寄存器分组策略优化核心算子的寄存器利用率；通过非对齐加载指令消除冗余内存操作，降低内存带宽占用 30%。

2.6.2 运行环境

(1) 硬件平台：

支持 RVV 扩展的 RISC-V 架构处理器（VLEN $\geq$ 128 位）。

软件依赖：

操作系统：Linux 内核 $\geq$ 5.4。

编译器：GCC $\geq$ 12.1 或 Clang $\geq$ 15.0。

构建工具：CMake $\geq$ 3.10，OpenCV 4.5+源码树。

2.6.3 限制和约束

(1) 技术条件限制

指令集兼容性差异：RVV 规范存在多版本指令集差异，例如 vle/vse 加载存储指令的语法和操作数格式不兼容。需通过预编译宏隔离版本实现动态指令映射，并在工具链层封装内联汇编转换接口，以解决 Clang 与 GCC 的语法差异问题。

硬件非法操作风险：寄存器分组策略需严格对齐物理寄存器索引，否则触发硬件异常。设计需内置运行时校验机制，检测非法配置时自动降级至 LMUL=1 并记录警告日志。

(2) 资源限制

物理寄存器数量约束：RVV 架构的寄存器合并特性受限于硬件物理寄存器数量。当 LMUL=8 时需占用 8 个连续寄存器，若可用寄存器不足则需动态降级分组策略。

内存带宽瓶颈：分块数据未对齐缓存行时，滑动窗口复用效率显著下降。需强制 64 字节内存对齐策略，并通过非对齐加载指令消除临时缓冲拷贝，降低 30%内存带宽占用。

(3) 工具链与开发环境约束

编译器依赖：需 GCC ≥ 12.1 或 Clang ≥ 15.0 。

仿真验证要求：QEMU 需覆盖 VLEN=128/256/512 三种配置场景，验证尾端截断精度误差 $\leq 1e-6$ 。仿真环境需预置 RISC-V 工具链及 OpenCV 4.5+源码树。

(4) 性能与质量边界

实时性要求：硬件感知层响应延迟需 $\leq 1\text{ms}$ ，超时触发动态降级机制。关键路径需预置标量后备路径，确保算法功能连续性。

(5) 兼容性约束

安全指令集白名单：严格限制仅使用 RVV v1.0 标准指令集，非法操作码触发 SIGILL 信号终止进程，防止未定义行为扩散。

2.6.4 设计原则和设计要求

(1) 模块独立性：

RVV 核心代码隔离于 `opencv/modules/core/src/hal/rvv` 目录，通过宏控制平台编译。

指令集兼容层与 LMUL 优化层解耦，支持独立替换或升级。

(2) 边界设计原则：

寄存器状态更新后插入内存屏障，保障多核一致性。

跨平台数据流通过 CMake 工具链文件抽象环境差异。

(3) 灵活性要求：

动态降级机制：连续检测到非法配置时，全局关闭 RVV 优化并切换至标量后备路径。

可配置 LMUL 阈值：通过接口允许手动调整分组策略。

(4) 可维护性要求：

QEMU 仿真覆盖：测试 $VLEN=128/256/512$ 三种配置，验证尾端截断精度误差 $\leq 1e-6$ 。

(5) 安全性与可靠性：

寄存器清零：敏感操作（如掩码寄存器）后显式清零，防止数据残留。

指令白名单：限制仅使用 RVV v1.0 标准指令，非法操作码触发 SIGILL 终止进程。

三、 总体设计

3.1 GCC 总体设计

3.1.1 总体架构

架构概述

RISC-V GNU Toolchain 是一套面向 RISC-V 架构的完整编译系统，主要用于将高级语言（如 C/C++）源码编译为目标平台上的可执行程序。其总体架构采用“分层解耦、模块可替换”的设计思想，主要包括：

前端(Frontend): 处理 C/C++/Fortran 等语言的语法和语义分析。

中间端（ Intermedia Code Generation ）：通用的中间表示（如 GIMPLE、RTL）优化与转换。

后端（ Backend ）：为 RISC-V 目标生成机器码，并进行指令调度与寄存器分配。

汇编器（ Assembler ）：将目标指令翻译为二进制.o 文件。

链接器（ Linker ）：将多个.o 文件链接成最终的可执行程序。

调试工具链：如 gdb，用于符号调试和二进制分析。

GCC Frontend	将源代码解析为 token 并生成抽象语法树 (AST)
GIMPLE/RTL Generation	中间代码生成, 优化 (如常量传播、死代码消除等)
RISC-V Backend	将 RTL 转换为 RISC-V 指令, 进行指令选择、寄存器分配等
Binutils (as/ld)	将编译器生成的 .s 汇编文件生成对应的二进制编码, 通过链接器链接 .o 文件生成 ELF 可执行文件
Newlib / Glibc	提供标准 C 运行时支持
GDB	支持调试 .elf 文件, 断点、单步执行等

与外部系统的关系

用户代码/项目代码: 通过命令行调用 GCC 进行构建。

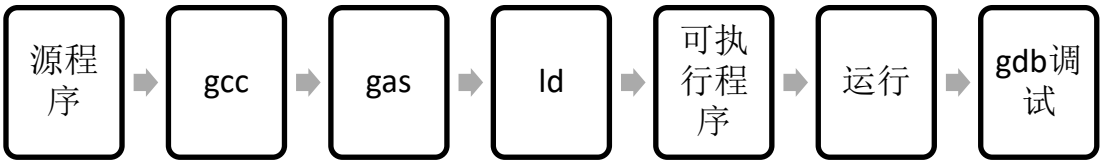
操作系统内核/裸机环境: 目标程序可运行于 Linux 系统、RTOS 或裸机。

外部构建系统: (如 CMake, Make) 与 CI/CD 工具 (如 Jenkins) 调用工具链并完成自动构建。

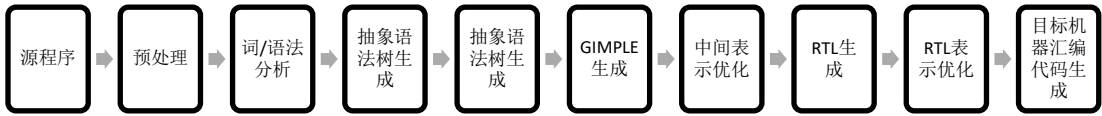
硬件仿真器/开发板 (如 QEMU/FPGA): 运行目标程序

3.1.2 逻辑处理流程

程序处理流程



GCC 编译器处理流程



3.1.3 功能分配

软件模块清单与功能性能说明：

表 1 GCC 功能模块表

模块编号	模块名称	主要功能	性能特点与设计 要求	与软件系统的 关系
M1	GCC 前端	解析源代码（C/C++/Fortran），生成中间表示（GIMPLE）	高兼容性（符合 ISO C/C++ 标准），编译语义准确，支持预处理宏	系统的入口模块，负责语言级输入处理，输出给优化器使用
M2	中间表示及优化器	执行语义保持的中间代码优化，如常量传播、死代码删	多层优化（GIMPLE → RTL），自动选择优化等级（-	提高生成代码质量，保持前后端解耦，是性能提升核心

		除、循环展开等	O1~O3)	
M3	GCC RISC-V 后端	将 RTL 映射为 RISC-V 指令，处理寄存器分配、调用约定、指令调度等	支持多种 RISC-V ISA 子集 (RV32/64, Zce, Zfinx, B 等)	面向目标架构生成机器代码，是系统定制能力的核心
M4	汇编器 (as)	将后端生成的汇编代码转换为目标文件(.o)	兼容 GNU 汇编语法，处理宏汇编、伪指令等	工具链的组成模块之一，配合 GCC 后端实现目标二进制代码生成
M5	链接器 (ld)	将多个目标文件和库链接为最终可执行程序(ELF 格式)	支持静态/动态链接、链接脚本、自定义节布局	构建过程的最后阶段，完成程序加载地址和符号解析
M6	C 语言库 libc (glibc/newlib)	提供运行时函数库支持，如 printf/malloc/文件 IO/系统	根据目标环境选择：glibc (Linux) 或 newlib (裸机)	支持程序正常运行，兼容标准 C 运行时规范

		调用封装		
M7	调试器 (gdb)	允许开发者在目标程序中设置断点、单步执行、检查变量、分析内存等	支持 DWARF 调试信息，支持远程调试 RISC-V 仿真器/硬件	提升可维护性和开发效率，是软件调试过程的核心工具
M8	Binutils 工具集	包含 objdump, readelf, nm, strip 等，用于查看、分析、精简二进制文件	支持各种 ELF 文件操作，高速响应，良好兼容性	提供程序可视化、符号查看、体积裁剪等辅助功能，是后期开发的重要工具
M9	构建系统 (Make/Autotools)	负责自动化配置、编译、构建 GCC 及相关工具	可跨平台构建，支持多种目标配置组合 (--with-arch=...)	管理构建流程、模块依赖，是开发和部署的基础设施

3.2 LLVM 总体设计

3.2.1 总体架构

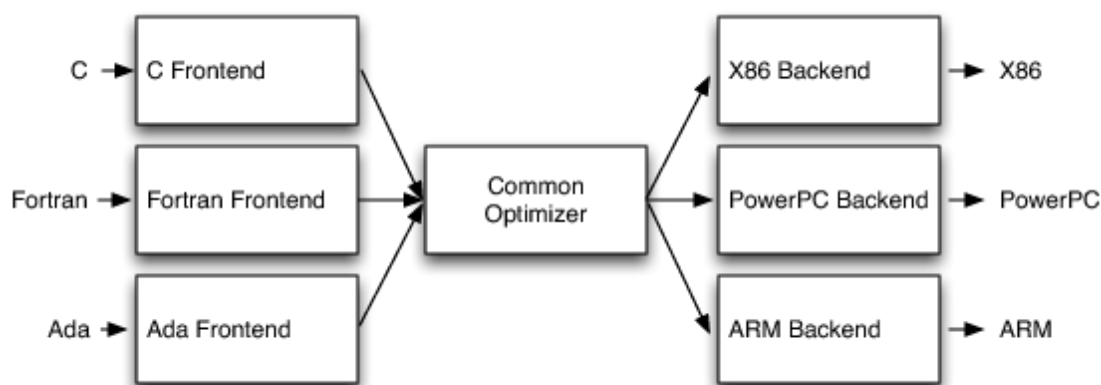


图 1 LLVM 总体架构图

LLVM 的总体架构采用经典的三阶段分层设计，以中间表示（IR）为核心枢纽，构建了一个高度模块化的编译框架。其架构可分为三个主要层次：

前端层负责将不同编程语言（如 C/C++/Rust/Swift 等）的源代码转换为统一的 LLVM 中间表示（IR）。这一层包含词法分析、语法分析、语义分析等传统编译前端组件，每个语言都有独立的前端实现，但都输出标准化的 LLVM IR。

中间层是 LLVM 的核心，由优化器和 LLVM IR 组成。LLVM IR 采用静态单赋值（SSA）形式的三地址码表示，兼具高级语言语义和低级硬件特征。优化器包含 300 多个可组合的优化 Pass，如内联优化、循环展开、死代码消除等，这些 Pass 通过管道方式依次对 IR 进行转换和优化。

后端层负责将优化后的 IR 转换为特定目标架构（如 x86/ARM/RISC-V/GPU 等）的机器代码。这一层实现指令选择、寄存

器分配、指令调度等目标相关优化，通过 TableGen 工具描述目标架构特征，使新硬件支持只需实现相应后端而无需修改优化器。

整个架构还包含完整的支持系统：ORC/MCJIT 实现即时编译，lld 链接器处理目标文件链接，LLDB 提供调试支持，compiler-rt 提供运行时库函数。这种设计使 LLVM 既可作为传统静态编译器，也能支持动态编译和跨语言优化。

本项目主要是为 LLVM 后端添加一些 RISC-V 的扩展指令集支持，让 RISC-V 后端能支持的指令集范围更加的广泛，适合更多的硬件和开发环境。

3.2.2 逻辑处理流程

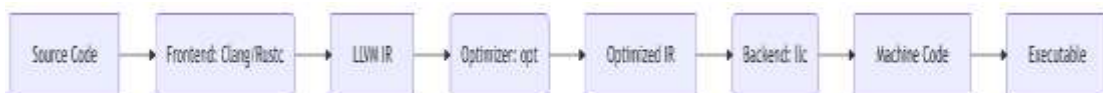


图 2 LLVM 逻辑处理流程图

LLVM 的前端处理阶段负责将高级语言源代码转换为 LLVM 中间表示(IR)。这一过程首先进行词法分析，将源代码分解为 Token 流，然后通过语法分析构建抽象语法树 (AST)，并进行语义检查确保程序逻辑正确。前端最终生成标准化的 LLVM IR，这是一种采用静态单赋值 (SSA) 形式的三地址码表示，兼具高级语言语义和低级硬件特征。不同语言的前端 (如 Clang、Flang、Rustc) 都遵循相同的 IR 生成规范，确保后续优化阶段的统一处理。

中间优化阶段对 LLVM IR 进行多层次的目标无关优化。优化器包含 300 多个可组合的优化 Pass，按特定顺序组成处理管道。这些

Pass 包括过程内优化（如常量传播、死代码消除）、过程间优化（如函数内联）以及循环优化（如循环展开、向量化）。优化过程会反复分析 IR 并应用转换规则，同时保持程序语义不变。这种基于 Pass 的设计允许开发者灵活定制优化流程，或插入自定义优化算法。

后端代码生成阶段将优化后的 IR 转换为目标机器代码。该阶段首先进行指令选择，将 IR 操作映射到目标架构的特定指令；然后通过寄存器分配算法管理有限的物理寄存器资源；最后进行指令调度以优化流水线效率。LLVM 采用目标描述语言（TableGen）来定义硬件特性，使新架构支持只需描述指令集和寄存器等信息，而无需修改核心优化逻辑。这种设计显著降低了移植到新硬件平台的工作量。

链接与执行阶段完成最终的可执行文件生成或即时运行。对于静态编译，LLVM 提供高效的 lld 链接器来处理目标文件链接；对于动态执行，ORC JIT 引擎支持即时编译和加载 IR 代码。LLVM 还集成了完整的调试支持（LLDB）和运行时检测工具（Sanitizers），形成从源码到运行的完整工具链。这种端到端的设计使 LLVM 既能作为传统静态编译器，也能支持动态语言实现和交互式开发。

3.2.3 功能分配

本项目的主要功能有：在汇编模块中添加对 RISC-V B 扩展的汇编支持，在汇编模块中添加对 RISC-V K 扩展的汇编支持，在汇编模块中添加对 RISC-V P 扩展的汇编支持，在汇编模块中添加对 RISC-V Zce 扩展的汇编支持和在汇编模块中添加对 RISC-V Zfinx 扩展的汇

编支持。

汇编模块中添加对 RISC-V B 扩展的汇编支持，主要是在汇编模块中添加对 RISC-V B 扩展的汇编支持，以确保能够正确识别、解析 B 扩展的汇编指令，并生成对应的机器码。

汇编模块中添加对 RISC-V K 扩展的汇编支持，主要是在汇编模块中添加对 RISC-V K 扩展的汇编支持，以确保能够正确识别、解析 K 扩展的汇编指令，并生成对应的机器码。

汇编模块中添加对 RISC-V P 扩展的汇编支持，主要是在汇编模块中添加对 RISC-V P 扩展的汇编支持，以确保能够正确识别、解析 P 扩展的汇编指令，并生成对应的机器码。

汇编模块中添加对 RISC-V Zce 扩展的汇编支持，主要是在汇编模块中添加对 RISC-V Zce 扩展的汇编支持，以确保能够正确识别、解析 Zce 扩展的汇编指令，并生成对应的机器码。

汇编模块中添加对 RISC-V Zfinx 扩展的汇编支持，主要是在汇编模块中添加对 RISC-V Zfinx 扩展的汇编支持，以确保能够正确识别、解析 Zfinx 的汇编指令，并生成对应的机器码。

3.3 V8 总体设计

3.3.1 总体架构

V8 具备清晰且层次化的模块结构，构建系统依托 GN、Ninja、Torque DSL 语言编译器、CSA 汇编器搭建编译基础；运行时支撑环境通过 snapshot 预编译和加载系统、底层平台接口封装系统、实用函

数库等子系统，为运行提供底层资源与服务，其中 Orinoco 和 Oilpan 垃圾回收系统及内置模拟器子系统保障内存管理与模拟能力；运行时集成解释器、基线编译器、Maglev 和 TurboFan 即时编译器，搭配 Wasm 执行引擎子系统（含 baseline 编译器、Wasm Turbofan 编译流水线）与 Tier 管理器和即时编译器分发器，实现高效代码执行；安全防护子系统借沙盒模块、Isolate 和 Native Context 模块筑牢安全防线；调试和剖析模块依靠 Debugger、Profiler、Trace、反汇编模块，助力开发调试与性能分析，各模块相互协同，支撑 V8 高效运行。

V8 的总体架构如下图所示：

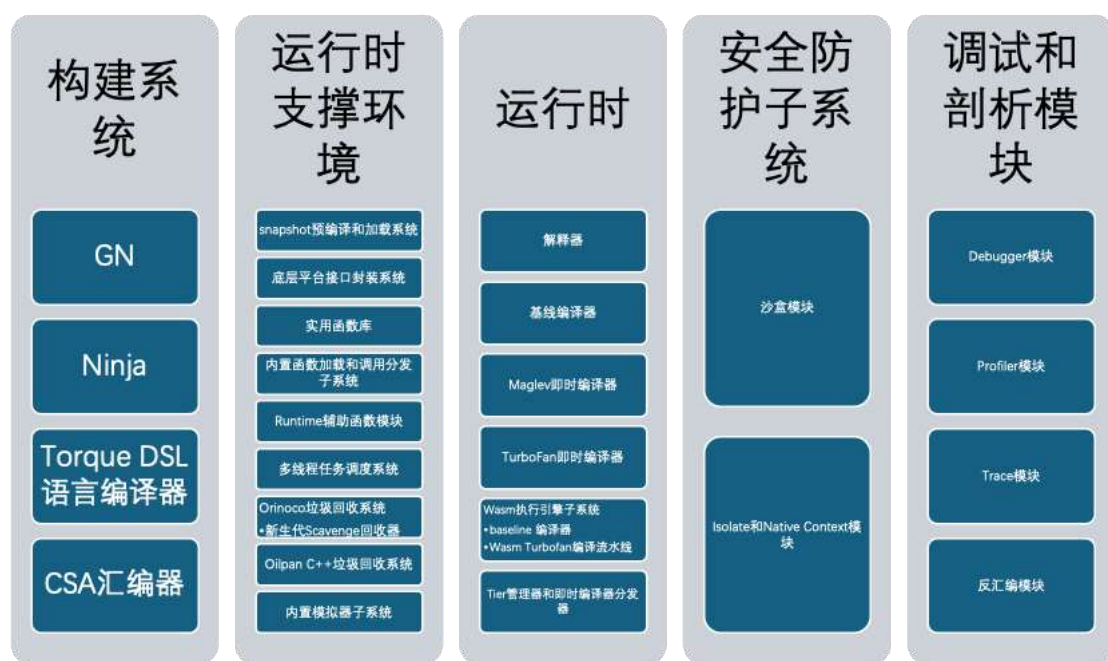


图 3 V8 总体架构图

3.3.2 逻辑处理流程

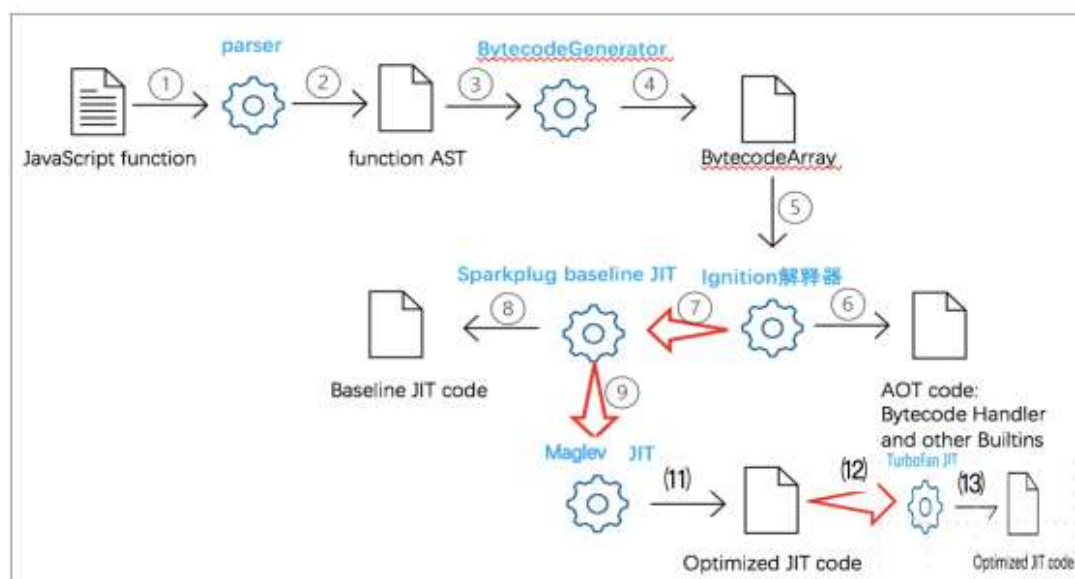


图 4 V8 逻辑处理流程图

3.3.3 功能分配

(1) 构建系统

1. GN

GN 是一种用于产生 Ninja 文件 meta-build system，Google Chromium 项目使用它产生构建配置。GN 的设计初衷是一种具有较小灵活性的构建任务描述语言，也就是说，设计者希望对同样的任务，不同的人员描述应该是相同的。GN 的完整参考文档见^[1]。本项目中需要修改 GN 配置文件来加入 RISC-V 构建和测试相关的功能。

2. Ninja

Ninja 是一种构建系统语言和执行系统，它类似于 Make 和 Makefile。Ninja 的作者是 Google 程序员 Evan Martin，Ninja 是为了解决 Chrome 的 Windows Visual Studio 工程移植到 Linux 上以后的构

建系统速度问题而诞生的。

Ninja 具有汇编级别的构建语句，语言元素简单、没有条件判断（相对应地，Make 具有条件判断语句，具有相对高级的语言级别），它设计初衷是利用其他更高级的 meta-build system，如 GN，来产生 build 文件，而不是手写，设计目标是“执行快速”。

目前，Ninja 被应用于在很多软件工程中，例如 Google 的 Chromium、Chrome，Android，LLVM 等。

Ninja 适用于大规模工程，对规模较小的工程的 build 速度影响很小。由于大规模工程手写 Ninja 文件是不太现实，Ninja 一般要配合 meta-build system（本项目中是 GN）使用。配置好 GN 后，由 GN 来产生 RISC-V 构建相关的 Ninja 文件。

(2) Torque DSL 语言编译器

V8 Torque 是一种语言，允许为 V8 项目做出贡献的开发者表达对虚拟机（VM）变更的意图，而无需过多关注无关的实现细节。该语言设计得足够简洁，便于将 ECMAScript 规范直接转化为 V8 的实现，同时又具有足够的强大功能，可以以稳健的方式表达底层的 V8 优化技巧，比如基于特定对象形状测试创建快速通道（fast-paths）。

(3) CSA 汇编器

CSA 即 CodeStubAssembler，是 V8 中的一个组件，它定义了一种建立在 TurboFan 后端之上的可移植汇编语言，是一个定制的、与平台无关的汇编器。CSA 使 V8 能以跨平台方式生成优化的汇编代码，这些代码集成到 V8 的内置函数、字节码处理程序中，提高了

JavaScript 代码的执行效率。CSA 为 V8 团队提供在底层快速、可靠地优化 JavaScript 功能的能力。它提供了手写汇编的优势，同时避免其脆弱和难以维护的问题，提高了团队的开发速度。例如，开发人员可以用跨平台的低级原语编写内置代码，指令选择器确保代码在各平台最优，无需成为各平台汇编语言专家；其接口的可选类型能保证操作值类型正确；自动进行寄存器分配，减少手动分配导致的细微错误；理解多种 ABI 调用约定，便于与 V8 其他部分互操作；代码为 C++，便于封装和复用常见代码生成模式；V8 测试框架支持直接从 C++测试 CSA 功能和相关内置函数，无需编写汇编适配器。

(4) 运行时支撑环境

提供基础功能支持，包括快照加载、平台接口封装、实用函数库和子系统，确保虚拟机高效稳定运行。

(5) snapshot 预编译和加载系统

实现预先编译快照，快速加载程序状态，加快启动速度，提高性能。

(6) 底层平台接口封装系统

封装不同硬件和操作系统的底层接口，保证跨平台的统一操作。

(7) 实用函数库

提供常用的工具函数，简化开发，提高效率。

(8) 内置函数加载和调用分发子系统

管理内置函数的加载与调用，优化执行流程。

(9) Runtime 辅助函数模块

提供运行时所需的辅助功能，支持内存管理和系统调用。

(10) 多线程任务调度系统

实现多线程任务调度，提高并发处理能力。

(11) Orinoco 垃圾回收系统

专为 V8 设计的垃圾回收机制，优化内存管理。

1. 新生代 Scavenge 回收器

年轻代垃圾收集，快速回收短生存期对象。

2. 老生代 Mark-Swept-Compact 回收器

老生代垃圾回收，使用 Mark-Sweep 和 Mark-Compact 算法

3. Oilpan C++垃圾回收系统

Oilpan 是一款基于追踪的垃圾回收器，这意味着它通过在标记阶段遍历对象图来确定存活对象，随后在清除阶段回收死亡对象。这两个阶段都可能与实际的 C++ 应用程序代码交错或并行运行。针对堆对象的引用处理是精确的，而对原生栈的处理则是保守的。这意味着 Oilpan 清楚堆上引用的位置，但对于栈，它必须在扫描内存时假设随机位序列代表指针。此外，当垃圾回收在没有原生栈的情况下运行时，Oilpan 还支持对某些对象进行压缩（整理堆碎片）。

(12) 内置模拟器子系统

模拟硬件行为，进行测试和调试

(13) 运行时

1. 解释器

以解释的方式来执行内部字节码。

2. 基线编译器

以直接翻译成本地代码和对应内置函数调用的形式来执行内部字节码。

3. Maglev 即时编译器

位于 Sparkplug 和 TurboFan 编译器之间的高速且优化的 Tier-2 JIT 编译器。

4. TurboFan 即时编译器

Tier-1 的基于类型反馈优化的 JIT 编译器。

5. Wasm 执行引擎子系统

支持 WebAssembly 运行。

6. baseline 编译器

提供快速且不优化的 WebAssembly 预编译路径。

7. Wasm Turbofan 编译流水线

用于 WebAssembly 的高级优化编译流程。

8. Tier 管理器和即时编译器分发器

管理不同优化阶段，动态调度编译任务。

(14) 安全防护子系统

确保执行环境安全，包括沙箱和隔离机制。

1. 沙盒模块

提供独立的执行上下文，增强安全和多任务能力。

2. Isolate 和 Native Context 模块

提供独立的执行上下文，增强安全和多任务能力。

(15) 调试和剖析模块

支持调试、性能分析和代码追踪。

1. Debugger 模块

实现断点调试和代码追踪。

2. Profiler 模块

采集性能数据，优化程序性能。

3. Trace 模块

追踪代码执行流程，分析性能瓶颈。

4. 反汇编模块

反汇编机器码，辅助调试和分析。

3.4 OpenJDK 总体设计

3.4.1 总体架构

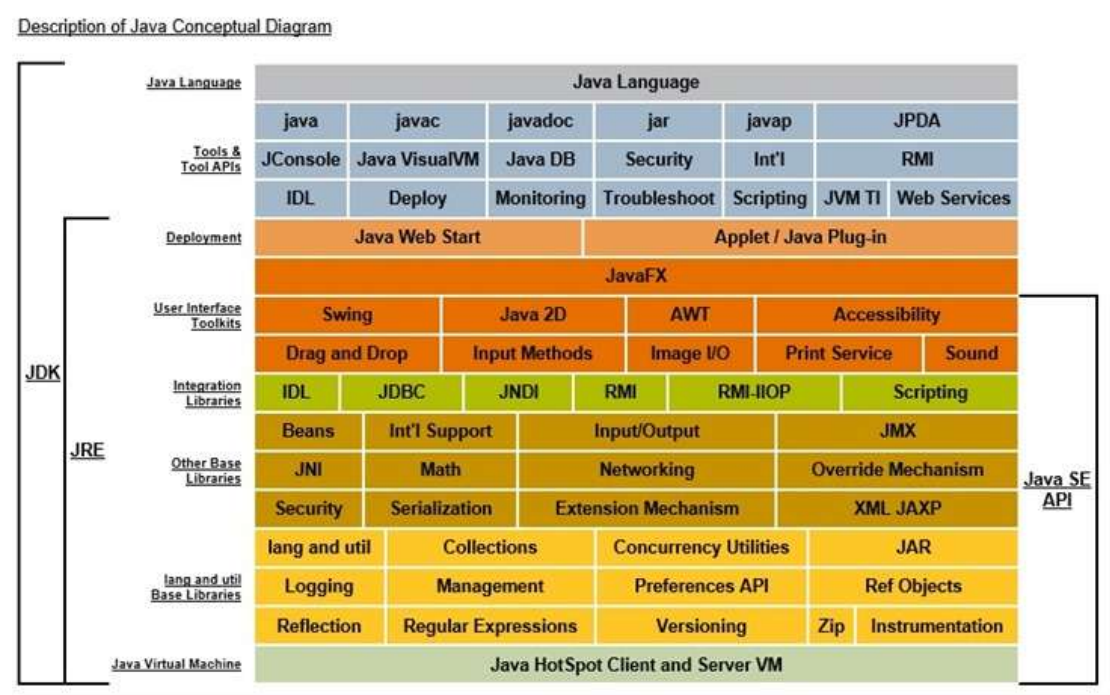


图 5 OpenJDK 总体架构

OpenJDK 的总体架构，可以分为：工具与工具 API 层、部署层、用户界面工具包、集成库、其他基础库、lang 和 util 基础库、Java 虚拟机等几个层面。该架构采用了模块化设计，各层功能明确；且具有跨平台性和扩展性。

工具与工具 API 层包含了 java、javac、javadoc、jar、javap、JPDA、JConsole、Java VisualVM、Java DB、Security、RMI、IDL、Deploy 等。部署层包含 Java Web Start 和 Applet/Java Plug-in。用户界面工具包包含 JavaFX、Swing、Java 2D、AWT、Accessibility、Drag and Drop、Input Methods、Image I/O 等。基础库包含了 IDL、JDBC、JNDI、RMI、RMI-IIOP 等。其他基础库包含了 Beans、Input/Output、JMX、JNI、Math 等。lang 和 util 基础库主要包含了 lang and util、Collections、Concurrency Utilities、JAR、Logging、Management、Preferences API 等。Java 虚拟机层则包含 HotSpot 和 Client/Server VM。

本项目主要聚焦于 Java 虚拟机层，针对 HotSpot 的相关内容进行移植。

3.4.2 逻辑处理流程

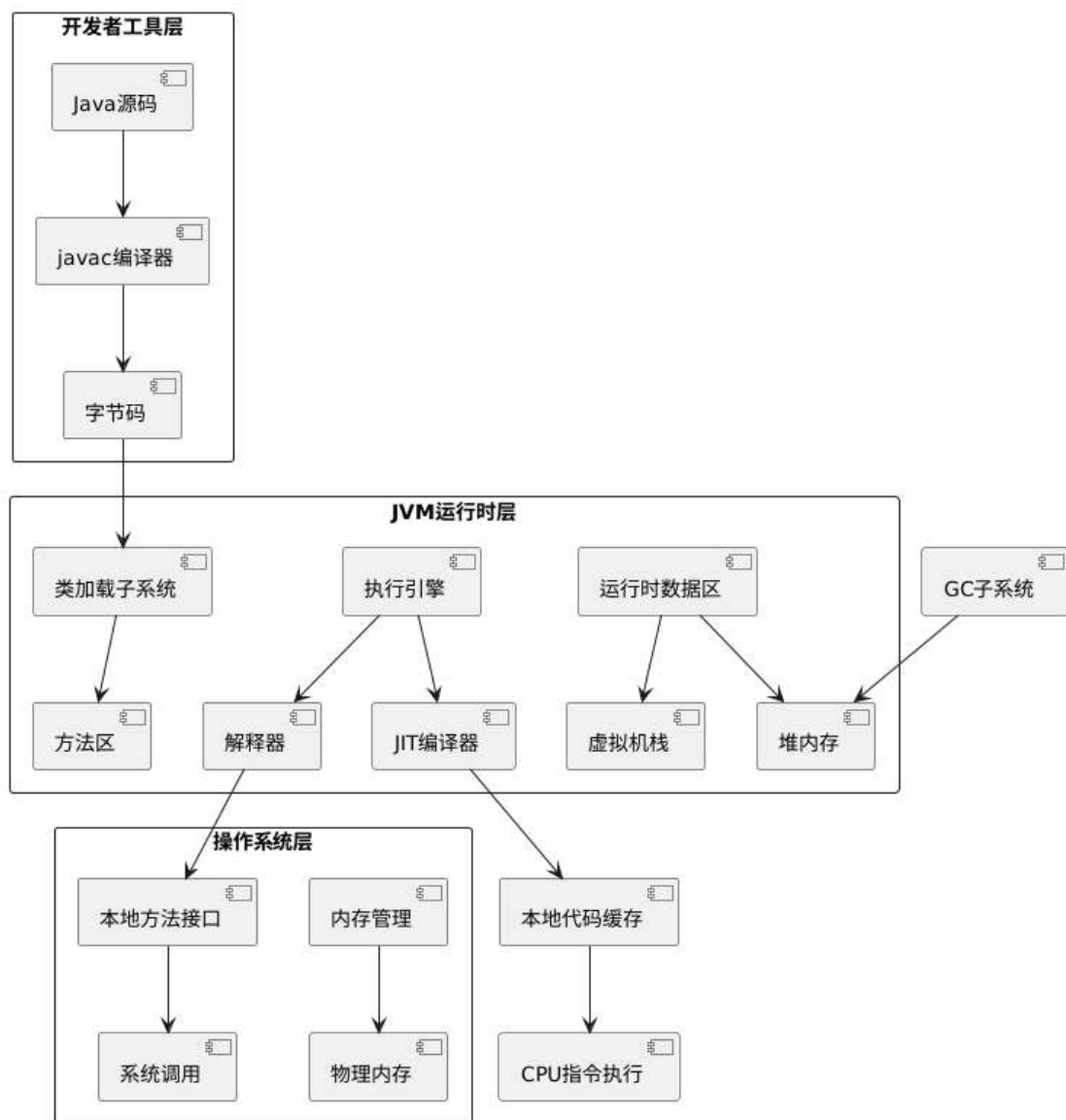


图 6 OpenJDK 全流程框架图

本项目属于 OpenJDK 针对 RV32G 和 RV64G 的移植项目，所以其处理流程都是和 OpenJDK 本身保持一致的，只不过是针对了 RV32G 指令集和 RV64G 指令集。本部分将介绍全流程框架和关键处理流程。

全流程框架图，通过 Java 程序首先通过 javac 编译工具，编译成为字节码。然后字节码通过类加载子系统，然后到达方法区。之后，

具体的方法会交给执行引擎来处理，执行引擎可以通过解释器或者 JIT 编译器两种路径来进行处理。解释器和 JIT 编译器，会有各自的路径，通过操作系统层或者是直接交给 CPU 进行执行。这里面还会涉及到运行时数据区，GC 子系统等内容，在聚焦流程的时候，也兼顾了整体的框架。

关键处理流程，是对核心关键流程单独进行梳理，将从 Java 程序到执行出结果整个流程进行了介绍，更加聚焦于关键流程的内部环节。Java 源代码通过语法分析之后生成 AST 抽象语法树，然后经过语义分析生成字节码，之后将字节码打包成 Class 文件，然后通过类加载、验证、准备、解析、初始化等环节，最终通过解释器或者 JIT 的方式，将程序执行得到结果。

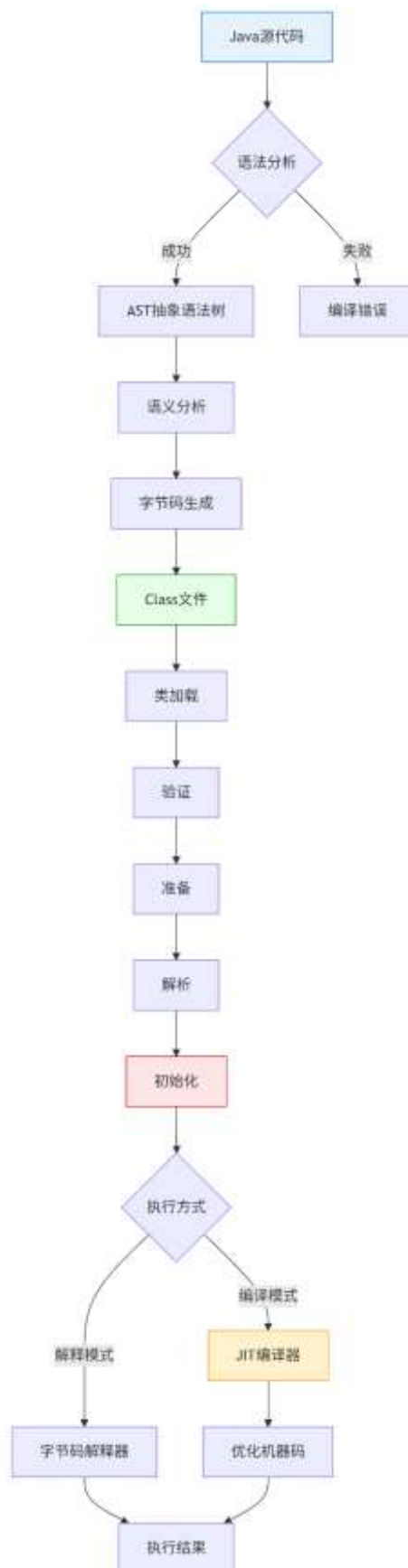


图 7 OpenJDK 关键处理流程图

3.4.3 功能分配

本项目主要的功能需求主要有：OpenJDK RV64G 支持现状调研、RV64G 开发板支持、OpenJDK 32 位版本调研、OpenJDK 32 位版本基线选择、OpenJDK RV32G 移植内容调研和 OpenJDK RV32G Zero 解释器移植。

OpenJDK RV64G 支持现状调研，对 OpenJDK 目前的开发版本中，RV64G 指令集的支持情况做详细的调研。RV64G 开发板支持，调研市场上目前已经量产的可以支持 RV64G 的开发板，然后使用 OpenJDK 测试其支持。这两个功能，是属于 OpenJDK RV64G 部分的功能，前者主要是做现状的调研，后者是对于具体开发板的适配。

OpenJDK 32 位版本调研，调研目前市场上使用比较多的 OpenJDK 32 位版本，为后续选择移植基线做好准备。OpenJDK 32 位版本基线选择，比较目前的 OpenJDK 32 位版本，从中选择一个版本作为移植的基线版本，为后续移植工作做好准备。OpenJDK RV32G 移植内容调研，调研 OpenJDK RV32G 移植所需要修改的内容范围，包含新增的文件以及原有代码的修改，确定整个移植所涉及到的范围。OpenJDK RV32G Zero 解释器移植，OpenJDK 的 Zero 解释器虽然具有很强的跨平台能力，不需要专门面向指令集的代码，但是针对 RISC-V RV32G，仍然需要对其进行适当的代码调整和测试，才能正常运行。这部分的功能都是属于 OpenJDK RV32G，属于解释器移植的前期准备工作和初步移植工作。

3.5 QEMU 总体设计

3.5.1 总体架构

(1) 分层架构

QEMU 的 RISC-V 模拟实现采用分层设计，新增的扩展指令集支持需融入现有框架，分为以下核心层级：

- 用户接口层：用户采用命令行启动 QEMU，通过设置 `-cpu` 参数选择处理器架构和扩展指令集。通过 `-s` 和 `-S` 参数连接 GDB 进行调试。在 GDB 中使用 `target remote` 命令连接到 QEMU 提供的调试端口。
- 仿真控制层：仿真控制层是 QEMU RISC-V 模拟器的核心调度中枢，负责协调虚拟硬件环境的运行。它通过事件循环 (Main Loop) 驱动指令执行、设备模拟和中断处理，管理虚拟设备（如定时器、中断控制器 PLIC、串口 UART）的交互，并在需要时触发指令处理层的翻译与执行。该层还处理用户输入的仿真控制命令（如暂停、快照），同时确保虚拟 CPU 状态、内存访问与设备 I/O 的同步，是连接用户接口层、指令处理层和硬件抽象层的枢纽。
- 指令处理层：指令处理层是 QEMU RISC-V 扩展指令集实现的核心模块，负责将二进制指令转换为可执行的模拟操作。该层首先通过解码模块（基于 `decodetree` 规则）解析 RISC-V 指令的二进制格式，随后由 TCG 翻译模块将指令转换为平台无关的中间代码 (TCG IR)，最终由执行逻辑模块模拟指令的原子行为（如寄存器

修改、内存访问)，并处理异常（如非法操作码）。同时，CSR 扩展模块和异常处理模块协同确保特权状态和中断的合规性，形成从指令解码到硬件状态更新的完整流水线。

- 硬件抽象层：硬件抽象层是 QEMU RISC-V 模拟器的底层基础架构，负责虚拟硬件环境的建模和状态管理。该层维护着虚拟 CPU 的核心状态（包括通用寄存器、浮点寄存器和控制状态寄存器 CSR），实现内存管理单元（MMU）的地址转换和 TLB 缓存，同时提供与物理设备交互的 I/O 总线接口。它通过标准化的 API 向上层屏蔽具体硬件差异，使指令处理层能专注于指令逻辑而无需考虑底层硬件细节，确保 RISC-V 架构的精确模拟，包括特权模式切换、内存保护机制和原子操作等关键硬件行为的正确实现。

QEMU 的四层分层架构示意图如图 8 所示。

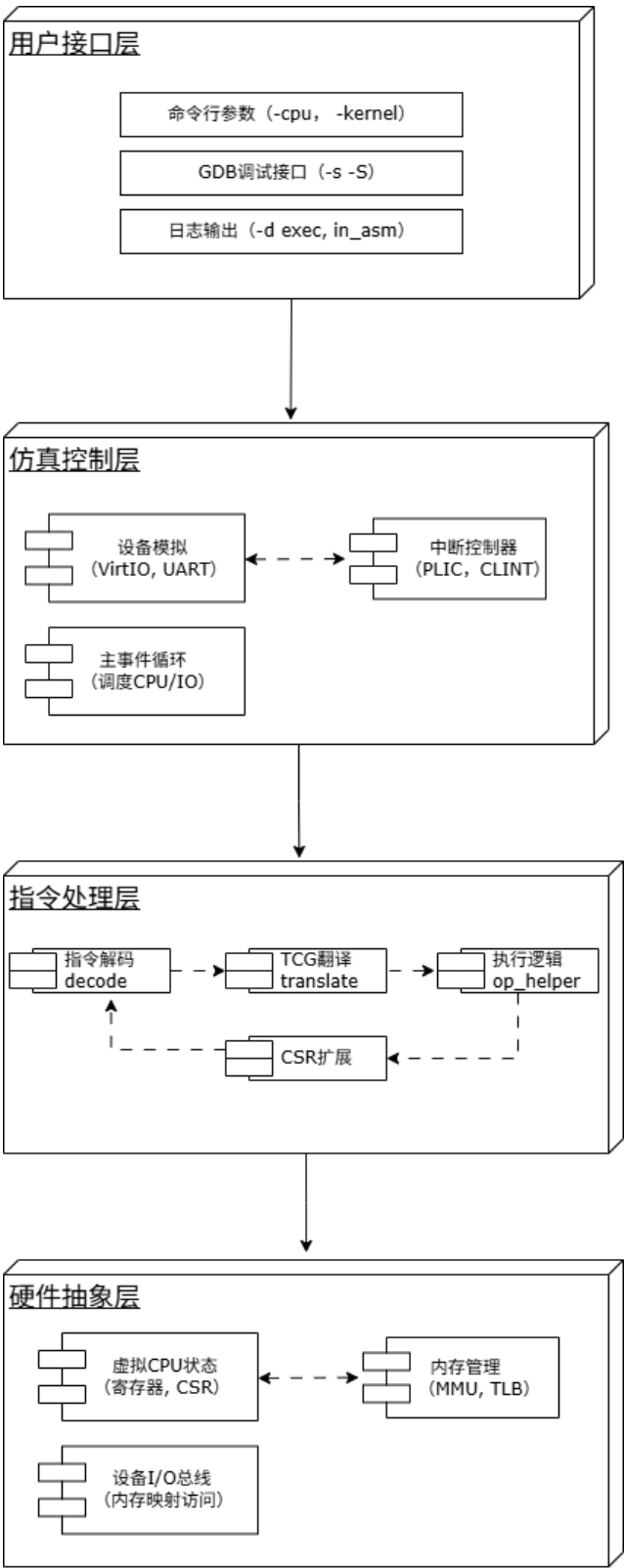


图 8 QMEU 分层架构示意图

(2) 核心模块划分

指令处理层是 QEMU RISC-V 扩展指令集实现的核心模块，负责

将二进制指令转换为可执行的模拟操作。核心模块构成了 QEMU RISC-V 扩展指令集支持的完整处理流水线：指令解码模块（`insn*.decode`）首先将二进制指令解析为内部表示；TCG 翻译模块（`translate.c`）随后将其转换为跨平台的中间代码（TCG IR）；执行逻辑模块（`op_helper.c`）具体实现指令的原子操作和副作用（如寄存器修改、内存访问）；CSR 扩展模块（`csr.c`）动态管理特权架构相关的控制状态寄存器；而异常处理模块（`cpu_helper.c`）则统一拦截非法操作和权限违规，确保虚拟 CPU 状态的完整性。这些模块通过标准化接口协同工作，在保持 RISC-V 规范精确性的同时，实现高效指令模拟。核心模块的简要介绍如表 2 所示。

表 2 指令处理层核心模块

模块	功能	关联文件/目录
指令解码	解析二进制指令为内部表示	target/riscv/insn*.decode
TCG 翻译	将指令转换为平台无关的中间代码	target/riscv/translate.c
执行逻辑	实现指令的原子操作、副作用	target/riscv/op_helper.c
CSR 扩展	管理控制状态寄存器	target/riscv/csr.c
异常处理	处理非法指令、权限错误	target/riscv/cpu_helper.c

(3) QEMU 软件拓扑结构

QEMU RISC-V 模拟器的软件拓扑结构呈现为三层四向的协同架构：最上层通过标准接口与外部工具链（RISC-V 测试套件、编译工

具链和 GDB 调试器）交互，形成验证闭环；核心模拟器采用分层设计，用户接口层接收命令并反馈调试信息，仿真控制层通过设备模型和中断控制器驱动仿真流程，硬件抽象层维护虚拟硬件状态；核心的指令处理层作为垂直通道，指令数据流依次流经解码、TCG 翻译和执行模块形成处理流水线，而 CSR 扩展和异常处理模块作为横向支撑系统贯穿各层，通过双向箭头与各层实时交互，确保状态同步和错误恢复。这种拓扑结构既保持了模块化开发的清晰边界，又通过纵横交错的交互网络实现了高效的指令模拟和系统仿真。QEMU RISC-V 模拟器的软件拓扑结构图如图 9 所示。

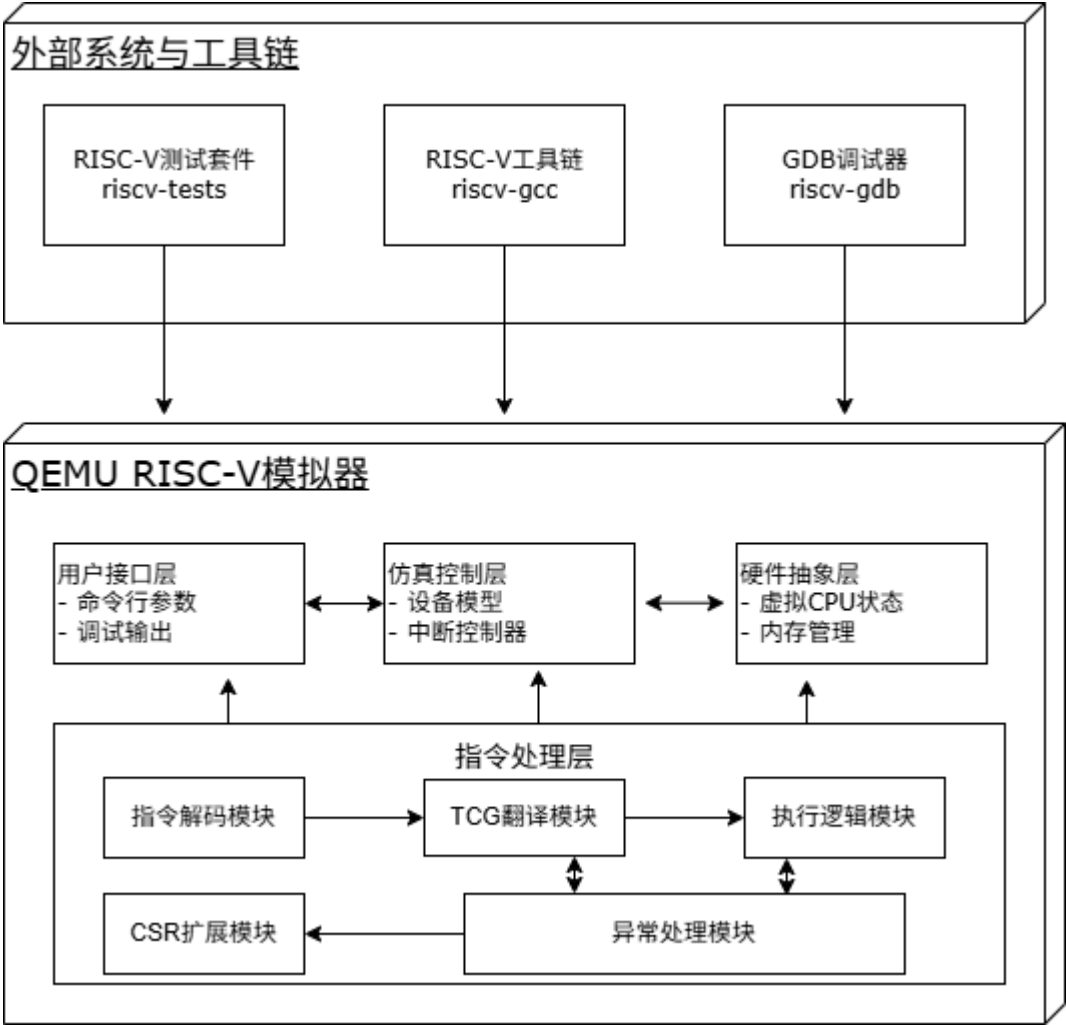


图 9 QEMU RISC-V 软件拓扑结构图

3.5.2 逻辑处理流程

图 10 展示了 QEMU RISC-V 扩展指令集支持项目的完整软件逻辑：从外部测试程序和调试工具的输入开始，指令流依次通过用户接口层、指令处理层（解码→TCG 翻译→执行）和硬件抽象层，形成单向处理流水线；同时异常处理和仿真控制层作为核心支撑模块，贯穿始终处理错误状态和设备交互，最终通过日志和调试接口形成验证闭环。整个架构体现了分层设计（用户接口/指令处理/硬件抽象）与横切关注点（异常/设备控制）的有机结合，既保证了指令模拟的高效精确，又为扩展开发提供了清晰的调试路径和性能分析接口。

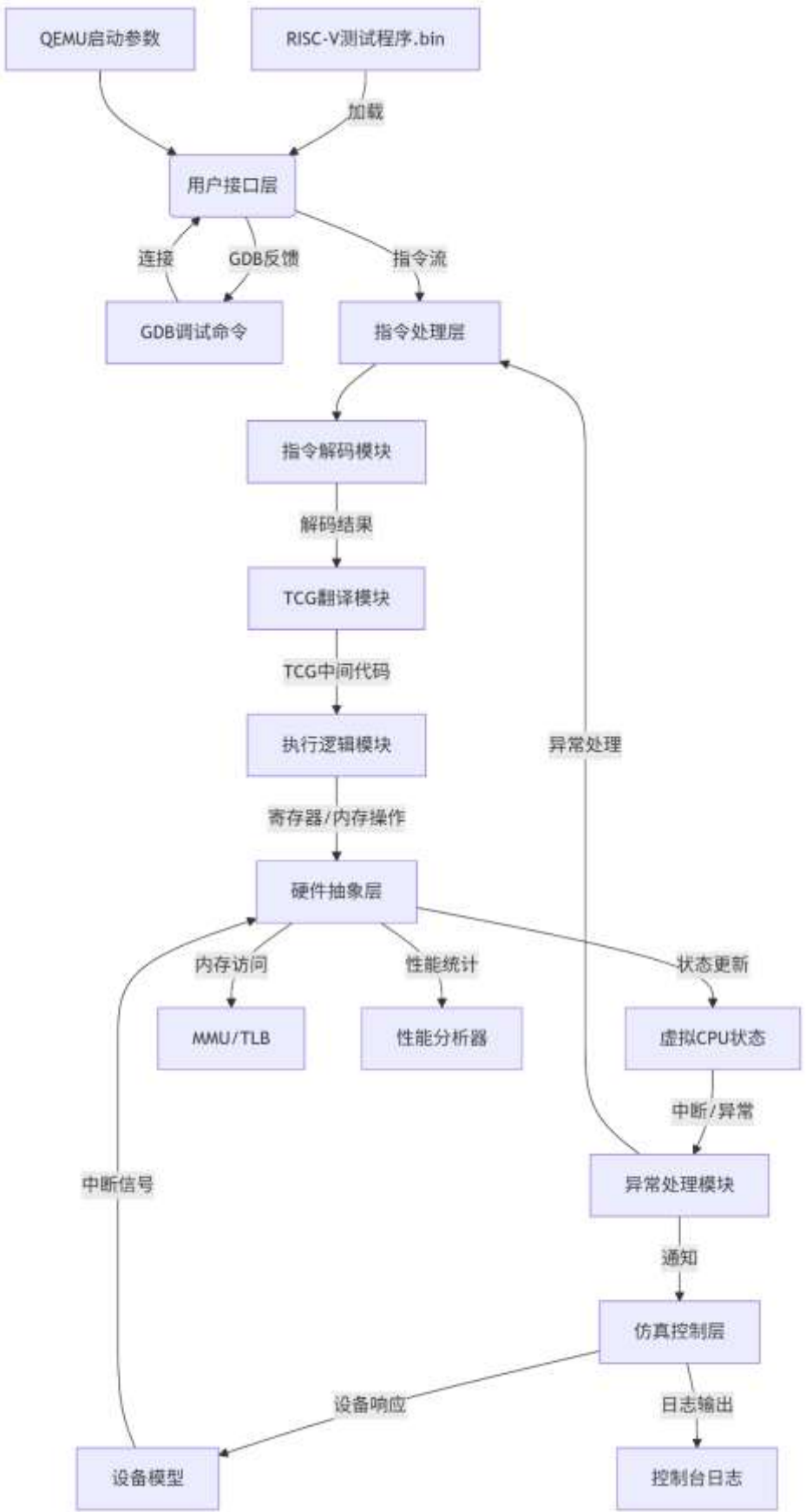


图 10 QEMU RISC-V 软件逻辑流程图

3.5.3 功能分配

各模块功能分配如下，涵盖主要功能、性能指标及其与系统的关系。

(1) 指令解码模块

功能	性能要求	与系统的关系
解析 RISC-V 二进制指令（32/16 位格式）	支持实时解码，单指令解码延迟 ≤ 100ns	作为指令处理流水线的起点，直接影响后续 TCG 翻译的正确性。
支持标准扩展（如 B、K、P、Zce、Zfinx）	兼容 RISC-V 官方 ISA 规范	确保新指令与现有指令集无冲突
生成内部中间表示（供 TCG 使用）	解码吞吐量 ≥ 10 ⁵ 指令/秒（单核）	与 TCG 翻译模块紧密耦合

关联文件有 target/riscv/insn32.decode, target/riscv/insn16.decode

(2) TCG 翻译模块

功能	性能要求	与系统关系
将指令转换为平台无关的 TCG 中间代码	单指令翻译开销 ≤ 基础指令的 1.2 倍	核心性能瓶颈，影响模拟器整体效率
优化中间代码（如常量折叠、死代码消除）	支持 JIT 加速（可选）	依赖 QEMU TCG 框架，需保持跨平台兼容性

处理指令间的依赖关系（如数据冒险）	确保原子操作的线程安全	与硬件抽象层的寄存器/内存接口交互
-------------------	-------------	-------------------

关联文件有 target/riscv/translate.c, tcg/tcg-op.c

(3) 执行逻辑模块

功能	性能要求	与系统关系
实现指令的原子操作（如 ALU、内存访问）	内存访问延迟 $\leq$ 主机原生访问的 5 倍	直接操作虚拟 CPU 状态，影响模拟精度
处理指令副作用（如 CSR 修改、异常触发）	支持并行指令执行（多核模拟）	与异常处理模块联动
支持向量/浮点等复杂指令的微操作拆分	向量指令吞吐量 ≥ 100 元素/周期（仿真）	需与硬件抽象层的 MMU 协同工作

关联文件有 target/riscv/op_helper.c, target/riscv/vector_helper.c

(4) CSR 扩展模块

功能	性能要求	与系统关系
管理控制状态寄存器	CSR 读写延迟 $\leq 200\text{ns}$	维护特权架构状态，影响中断/异常行为
实现特权级切换（U/S/M 模式）	支持原子 CSR 操作（如 CSRRC）	与异常处理模块共同确保安全隔离
提供自定义 CSR 的注册接口	兼容 RISC-V 特权规范 v1.12	硬件抽象层的关键组成部分

关联文件是 target/riscv/csr.c, target/riscv/cpu.h

(5) 异常处理模块

功能	性能要求	与系统的关系
捕获非法指令、内存访问错误等异常	异常响应时间 $\leq 1s$	横跨所有层级，确保系统鲁棒性
触发中断和异常处理流程（如 ECALL、BREAK）	支持嵌套异常处理	与仿真控制层的中断控制器（PLIC）交互
提供调试接口（如 GDB 的 catch exception）	兼容 riscv-tests 的异常测试用例	用户调试功能的核心依赖

关联文件有 target/riscv/cpu_helper.c, target/riscv/debug.h

(6) 硬件抽象层

功能	性能要求	与系统的关系
维护虚拟 CPU 状态（寄存器、内存映射）	状态同步延迟 $\leq 100ms$	所有指令执行的最终操作目标
实现 MMU/TLB 的地址转换和权限检查	页表查询吞吐量 $\geq 10^5/秒$	保护模式运行的基础设施
提供设备 I/O 总线接口（如 VirtIO、UART）	设备中断延迟 $\leq 1s$	连接仿真控制层的设备模型

关联文件有 target/riscv/cpu.c, hw/riscv/virt.c

(7) 仿真控制层

功能	性能要求	与系统的关系
调度 CPU 执行和设备模型（事件循环）	支持多核并行仿真	系统的中枢调度器
管理中断控制器（PLIC/CLINT）	中断分发延迟≤1s	连接硬件抽象层与外部设备
提供快照（Snapshot）和迁移（Migration）支持	快照生成时间≤2s	用户接口层的功能依赖

关联文件有 hw/intc/riscv_plic.c, accel/tcg/cpu-exec.c

(8) 用户接口层

功能	性能要求	与系统的关系
解析命令行参数(如 -cpu rv64,bext=true)	参数解析时间≤1ms	系统启动的入口点
支持 GDB 远程调试和日志输出 (-d exec)	调试命令响应时间 ≤50μs	开发者核心交互接口

关联文件是 softmmu/vl.c, gdbstub/gdbstub.c

3.6 OpenCV

3.6.1 总体架构

本系统采用分层硬件抽象架构，通过三级垂直分层实现算法逻辑与硬件指令集的解耦。顶层算法接口层承接 OpenCV 通用 Intrinsic 调用，将核心算子请求分发至中间调度层；该层动态解析硬件向量参数，

基于运行时寄存器状态生成最优循环分块策略，并调用底层 RVV 指令集适配层执行向量化计算。架构通过编译时宏隔离与运行时降级开关双机制保障跨平台兼容性：当检测到 RVV 硬件不可用或指令集版本冲突时，自动切换至通用 SIMD 后端，确保算法功能连续性。外部系统依赖 CMake 工具链抽象平台差异。

● 外部依赖：

编译器工具链：依赖 GCC ≥ 12.1 或 Clang ≥ 15.0 。

硬件平台：需 RVV 扩展支持。

仿真环境：QEMU 覆盖 VLEN=128/256/512 验证场景，确保尾端截断精度误差 $\leq 1e-6$ 。

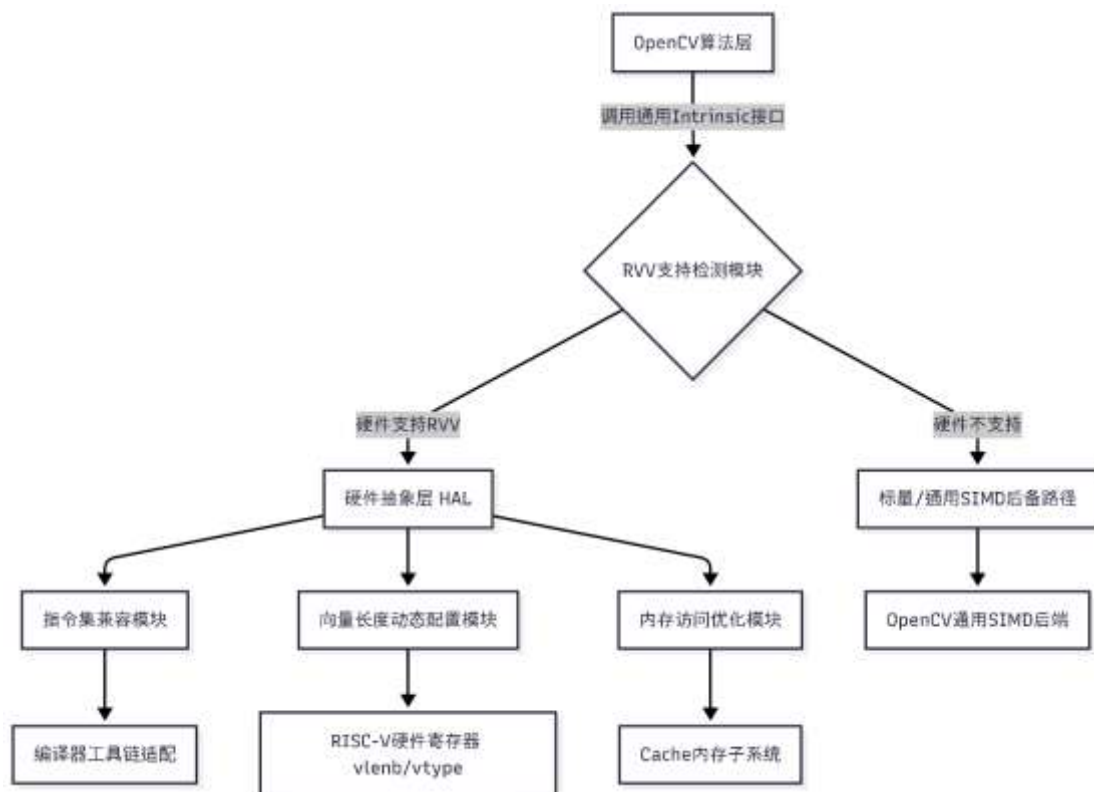


图 11 OpenCV 总体架构图

3.6.2 逻辑处理流程

本系统的核心逻辑流程围绕通用 Intrinsic 抽象层与 RVV 硬件适配层的交互展开，采用动态配置-批量处理-异常降级三层模型，实现可变长向量操作的统一调度。以下流程适用于所有通过通用 Intrinsic 接口调用的图像处理算子（如像素转换、矩阵转置等）：

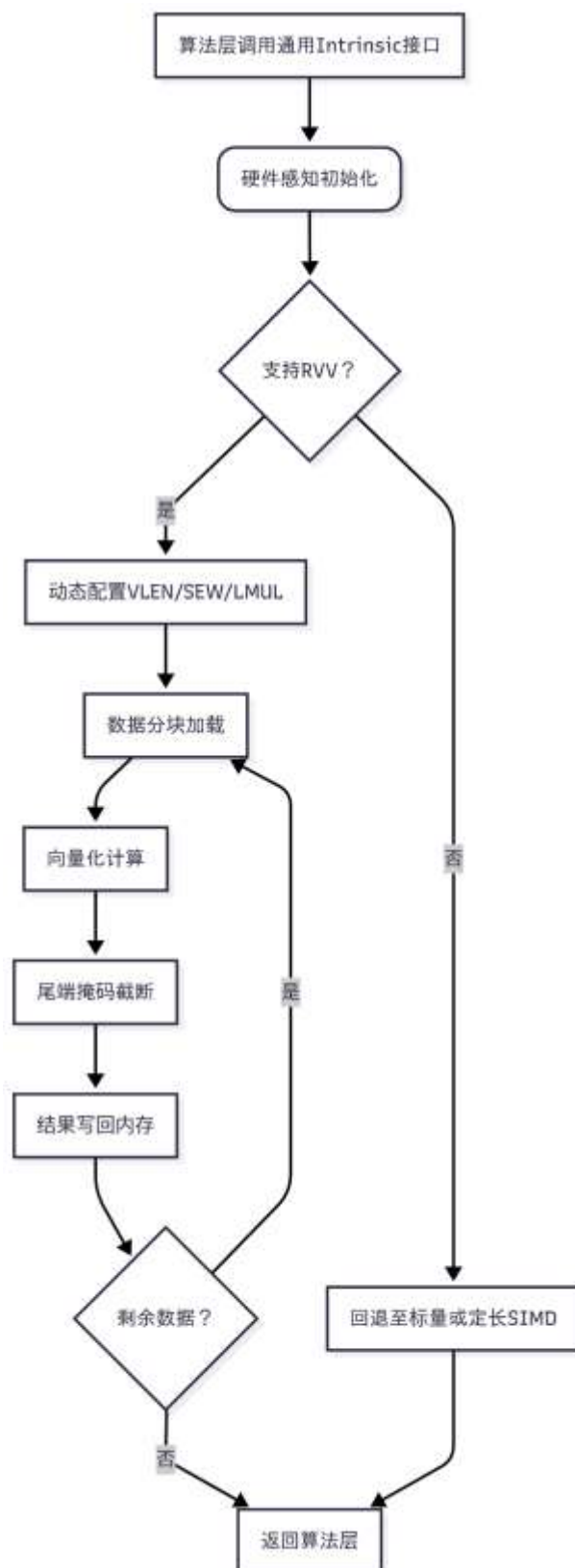


图 12 OpenCV 逻辑处理流程图

(1) 流程阶段详解

● 硬件感知初始化

通过 `vsetvl` 指令读取硬件寄存器 (`vlenb`、`vtype`)，动态获取当前平台的向量长度 (VLEN) 和元素位宽 (SEW)。

根据数据总量与 VLEN 计算初始分块大小，并设置 LMUL 分组策略。

- 数据分块加载

使用 `vlseg` 系列指令批量加载非连续内存数据，通过 Segment Load 机制将多通道数据拆分至独立向量寄存器。

内存地址强制 64 字节对齐，规避 Cache 未命中导致的性能衰减。

- 向量化计算

调用 RVV 原生函数执行计算，运算粒度由 SEW 自动适配。

掩码寄存器支持条件执行，非活跃元素跳过计算以降低功耗。

- 尾端掩码截断

剩余数据量不足 VLEN 时，通过 `vsetvli` 动态缩小向量长度，启用尾端掩码 (Tail Agnostic) 避免越界操作。

掩码策略由 `vta/vma` 寄存器控制，默认采用 Agnostic 模式，减少寄存器重命名开销。

- 异常降级与回退

连续检测非法配置时，全局关闭 RVV 优化并切换至标量后备路径，确保功能连续性。

硬件不支持非对齐访问时，自动插入缓冲拷贝操作并记录性能警告。

(2) 流程特性与当前阶段限制

- 动态适应性：

VLEN/SEW 运行时绑定机制，使同一 Intrinsic 接口可透明适配多种硬件规格。
- 与手工优化的边界：

当前流程完全通过通用 Intrinsic 接口实现，未引入算子级手工优化，性能加速依赖编译器自动向量化能力。
- 性能监测闭环：

集成 Perf 性能计数器分析向量指令利用率，Valgrind 监测内存带宽，数据反馈至 LMUL 策略选择模块实现动态调优。

3.6.3 功能分配

各模块功能与性能指标如下表所示，覆盖硬件适配、指令兼容及资源优化三大核心领域：

模块名称	主要功能	性能指标	与系统的关系
硬件感知模块	运行时读取 vlenb/vsew 寄存器，初始化向量配置参数	响应延迟≤1ms	驱动算子调度层的循环分派策略
指令集兼容模块	通过预编译宏切换指令实现，封装 Clang/GCC 内联汇编差异	指令映射覆盖率 100%	确保跨编译器兼容性，

模块名称	主要功能	性能指标	与系统的关系
			隔离硬件差异
LMUL 优化模块	动态计算最优分组策略，强制寄存器索引对齐硬件约束	加速比 ≥ 40%	提升寄存器利用率，降低分块计算延迟
内存访问优化模块	采用非对齐加载指令消除临时缓冲拷贝，结合 64 字节缓存行对齐策略	内存带宽占用降低 30%	减少数据搬运开销，优化 Cache 命中率
动态降级模块	连续检测 vill=1 时关闭 RVV 优化，回退至标量后备路径	降级触发延迟 ≤ 10 μs	保障算法功能连续性，隔离硬件故障

- 模块协同关系：
- 性能关键路径：LMUL 优化模块与内存访问模块协同作用于核心算子，通过寄存器分组和滑动窗口指令实现计算-内存访问平衡。
 - 安全边界：动态降级模块监控硬件感知模块的输出，非法操作触

发进程级防护。

- 可维护性设计：支持 QEMU 仿真实验，降低版本迁移成本。

四、 附录

[1] GNU 编码标准

<https://www.gnu.org/prep/standards/>

[2] RISC-V 指令集规范

<https://github.com/riscv/riscv-isa-manual>

[3] RISC-V psABI 规范

<https://github.com/riscv-non-isa/riscv-elf-psabi-doc>

[4] RISC-V 调用约定

<https://github.com/riscv-non-isa/riscv-toolchain-conventions/>

[5] RISC-V GNU Toolchain 仓库

<https://github.com/riscv-collab/riscv-gnu-toolchain>

[6] RVV v1.0 官方规范文档

<https://github.com/riscv/riscv-v-spec>

[7] RVV Intrinsic 编程指南

<https://github.com/riscv/rvv-intrinsic-doc>

[8] GCC RISC-V 支持手册

<https://gcc.gnu.org/onlinedocs/gcc/RISC-V-Options.html>

[9] Clang RISC-V 向量化指南

<https://clang.llvm.org/docs/RISCVVectorSupport.html>

[10] OpenCV Universal Intrinsic 官方文档

https://docs.opencv.org/master/d91/group_core_hal_intrin.html

[11] OpenCV 官方编程规范

https://github.com/opencv/opencv/wiki/Coding_Style_Guide

[12] OpenCV RISC-V 移植教程

<https://github.com/opencv/opencv/wiki/OpenCV-RISC-V>

[13] QEMU RISC-V 模拟器手册

<https://www.qemu.org/docs/master/system/riscv/virt.html>